RV College of Engineering®
Mysore Road, RV Vidyaniketan Post, Bengaluru - 560059, Karnataka, India

Go, change the world

Intelligent Clinical Document Processing and SNOMED Mapping System

A Major Project Report (AI481P)

Submitted by,

Aryan Sinha	**1RV22AI009**
Kushagra Aatre	**1RV22AI025**
Nischitha P	**1RV22AI031**
Shreya M	**1RV22AI054**

Under the guidance of

Dr. Somesh Nandi
Assistant Professor
Dept. of AIML
RV College of Engineering

.

In partial fulfillment of the requirements for the degree of Bachelor of Engineering in

Artificial Intelligence and Machine Learning

2025-26

RV College of Engineering®, Bengaluru

(*Autonomous institution affiliated to VTU, Belagavi*)

Department of Artificial Intelligence and Machine Learning

CERTIFICATE

Certified that the major project (AI481P) work titled ***Intelligent Clinical Document Processing and SNOMED Mapping System*** is carried out by **Aryan Sinha** (**1RV22AI009**), **Kushagra Aatre** (**1RV22AI025**), **Nischitha P** (**1RV22AI031**) and **Shreya M** (**1RV22AI054**) who are bonafide students of RV College of Engineering, Bengaluru, in partial fulfillment of the requirements for the degree of **Bachelor of Engineering** in **Artificial Intelligence and Machine Learning** of the Visvesvaraya Technological University, Belagavi during the year 2025-26. It is certified that all corrections/suggestions indicated for the Internal Assessment have been incorporated in the major project report deposited in the departmental library. The major project report has been approved as it satisfies the academic requirements in respect of major project work prescribed by the institution for the said degree.

Guide	Head of the Department	Principal
Dr. Somesh Nandi	Dr. B. Satish Babu	Dr. K. N. Subramanya

External Viva

Name of Examiners | Signature with Date

1.

2.

DECLARATION

We, **Aryan Sinha**, **Kushagra Aatre**, **Nischitha P** and **Shreya M** students of eighth semester B.E., Department of Artificial Intelligence and Machine Learning, RV College of Engineering, Bengaluru, hereby declare that the major project titled '**Intelligent Clinical Document Processing and SNOMED Mapping System**' has been carried out by us and submitted in partial fulfilment for the award of degree of **Bachelor of Engineering** in **Artificial Intelligence and Machine Learning** during the year 2025-26.

Further we declare that the content of the dissertation has not been submitted previously by anybody for the award of any degree or diploma to any other university.

We also declare that any Intellectual Property Rights generated out of this project carried out at RVCE will be the property of RV College of Engineering, Bengaluru and we will be one of the authors of the same.

Place: Bengaluru

Date:

Name	Signature

1. Aryan Sinha(1RV22AI009)

2. Kushagra Aatre(1RV22AI025)

3. Nischitha P(1RV22AI031)

4. Shreya M(1RV22AI054)

ACKNOWLEDGEMENTS

We are indebted to our guide, **Dr. Somesh Nandi**, Assistant Professor, Department of AIML, RVCE for the wholehearted support, suggestions and invaluable advice throughout our project work and also helped in the preparation of this thesis.

We also express our gratitude to our panel member **Prof. Rajesh R. M.**, Assistant Professor, Department of AIML, RVCE for his invaluable comments and suggestions during the phase evaluations.

Our sincere thanks to the project coordinators **Prof. Rushikesh Anil Padaki**, Assistant Professor, Department of AIML and **Prof. Harshitha V**, for their timely instructions and support in coordinating the project.

Our gratitude to **Prof. Narashimaraja P**, Department of ECE, RVCE for the organized latex template which made report writing easy and interesting.

Our sincere thanks to **Dr. B. Satish Babu**, Professor and Head, Department of Artificial Intelligence and Machine Learning, RVCE for the support and encouragement.

We express sincere gratitude to our beloved Professor and Vice Principal, **Dr. Geetha K S**, RVCE and Principal, **Dr. K. N. Subramanya**, RVCE for the appreciation towards this project work.

We thank all the teaching staff and technical staff of Artificial Intelligence and Machine Learning department, RVCE for their help.

Lastly, we take this opportunity to thank our family members and friends who provided all the backup support throughout the project work.

ABSTRACT

The healthcare sector generates vast quantities of unstructured clinical data through documents such as referral letters and discharge summaries. Manual processing of these records is time-consuming and error-prone, creating significant bottlenecks in clinical workflows. This project addressed the challenge of automated medical document understanding and Systematized Nomenclature of Medicine — Clinical Terms (SNOMED CT) coding within a template-free intelligent extraction framework.

The domain of clinical Natural Language Processing (NLP) has expanded rapidly with the adoption of Electronic Health Records (EHR) and the growing volume of clinical documents requiring structured coding. However, many existing systems rely on rigid, template-dependent approaches that fail to adapt to diverse document layouts commonly encountered in real-world National Health Service (NHS) environments. This limitation often results in incomplete or inaccurate code mappings.

The proposed system, developed as an Intelligent Clinical Document Processing System (ICDPS), aimed to: (i) create an intelligent ingestion module for PDF and image-based clinical documents; (ii) develop a non-template-based extraction engine for diverse layouts; (iii) automate clinical summarization and categorization into predefined medical fields; and (iv) normalize free-text clinical concepts to SNOMED CT codes.

The methodology followed an iterative Software Development Life Cycle (SDLC) consisting of requirements analysis, design, implementation, testing, and deployment. OCR pipelines handled image-based inputs, while direct extraction processed PDFs. Transformer-based NLP models were employed for Named Entity Recognition (NER) and classification of entities such as Diagnosis, Medication, Procedure, and Allergy. A confidence-ranked SNOMED CT suggestion interface assisted users in selecting suitable codes.

The system achieved clinical entity extraction accuracy exceeding 88% and SNOMED CT mapping precision of 85% on the test dataset, satisfying project requirements. The resulting system demonstrated potential to reduce administrative burden, accelerate structured record generation, and improve the quality of medical coding in clinical environments.

CONTENTS

LIST OF FIGURES

LIST OF TABLES

Chapter 1

Introduction

CHAPTER 1

INTRODUCTION

Clinical document processing and automated medical coding have emerged as important research areas due to the rapid growth of unstructured healthcare data and the increasing need for efficient information management. Understanding the challenges associated with extracting meaningful information from heterogeneous clinical documents is essential for developing intelligent healthcare solutions capable of supporting clinical workflows. The proposed Intelligent Clinical Document Processing System (ICDPS) with SNOMED CT Coding addresses these challenges through the integration of document understanding, natural language processing, and medical terminology mapping techniques. The chapter presents the background and significance of the problem domain, discusses the project objectives and scope, examines existing research through a literature review, identifies research gaps, and outlines the methodology and organization of the overall work.

1.1 Introduction

Clinical document processing represents one of the most labour-intensive and administratively demanding activities in modern healthcare systems. Hospitals and healthcare institutions generate thousands of documents daily, including referral letters, discharge summaries, outpatient records, diagnostic reports, and clinical notes. These documents contain valuable patient information necessary for diagnosis, treatment planning, and long-term healthcare management. However, much of this information exists in unstructured free-text form, making efficient processing and analysis difficult.

Historically, clinical coding and information extraction have been performed manually by trained healthcare professionals and clinical coders. While manual approaches provide flexibility, they are time-consuming, expensive, and susceptible to inconsistencies arising from subjective interpretation and human error. As healthcare systems increasingly adopt Electronic Health Records (EHRs), the volume of digital clinical data has expanded significantly, creating an urgent need for intelligent automation methods capable of processing information efficiently.

Recent developments in Artificial Intelligence (AI), Natural Language Processing (NLP), and deep learning technologies have created opportunities for transforming clinical

document processing workflows. Advanced transformer-based architectures can understand contextual relationships in medical text and extract clinically meaningful information from heterogeneous document formats.

The proposed Intelligent Clinical Document Processing System (ICDPS) with SNOMED CT Coding aims to address these challenges through a template-independent framework capable of processing PDF and image-based clinical documents. The system integrates Optical Character Recognition (OCR), Named Entity Recognition (NER), and ontology-based clinical coding to convert unstructured clinical narratives into structured and meaningful information representations suitable for healthcare applications.

1.1.1 Terminology and Definitions

The development of the Intelligent Clinical Document Processing System (ICDPS) involves several concepts and technical terms from healthcare informatics, Natural Language Processing (NLP), Artificial Intelligence (AI), and clinical coding systems. Understanding these terms is essential for interpreting the design and implementation aspects of the proposed system. The important terminology used throughout this report is presented below.

- **Clinical Document:** A healthcare-related document containing patient information, medical history, diagnoses, prescriptions, laboratory reports, referrals, discharge summaries, and clinical observations.
- **Natural Language Processing (NLP):** A branch of Artificial Intelligence concerned with enabling computers to understand, process, analyze, and generate human language.
- **Optical Character Recognition (OCR):** A technology used for converting text embedded within scanned documents or images into machine-readable textual information.
- **Named Entity Recognition (NER):** A Natural Language Processing task used to identify and classify specific entities from text into predefined categories such as diseases, medications, procedures, symptoms, and patient details.
- **SNOMED CT:** Systematized Nomenclature of Medicine – Clinical Terms, a comprehensive clinical terminology system used to represent medical concepts in a stan-

dardized manner.

- **Electronic Health Record (EHR):** A digital version of a patient's medical history that includes clinical data collected and maintained across healthcare systems.
- **Transformer Model:** A deep learning architecture that uses attention mechanisms to capture contextual relationships within sequential data and is widely used in modern NLP systems.
- **Intelligent Clinical Document Processing System (ICDPS):** The proposed framework designed to automate extraction of clinical information from heterogeneous healthcare documents and map extracted information to standardized SNOMED CT codes.

1.2 Overview of the Project Topic

The domain of intelligent clinical document processing sits at the intersection of healthcare informatics, natural language processing, and machine learning. The ability to automatically extract structured information from clinical narratives has profound implications for patient safety, clinical efficiency, and healthcare analytics. This section provides a broad overview of the domain from global, societal, and problem-specific perspectives.

1.2.1 Global Scenario

Globally, healthcare systems are undergoing a digital transformation driven by EHR adoption, telemedicine, and data-driven clinical decision support. According to reports from the World Health Organization (WHO) and major health informatics bodies, the volume of clinical data generated per patient encounter has grown exponentially over the past decade. In the United Kingdom, the NHS processes over one million patient interactions every 36 hours, generating vast quantities of unstructured clinical text. Similar volumes are observed in the United States, where the Centers for Medicare and Medicaid Services (CMS) mandates structured clinical documentation for reimbursement purposes.

Research in clinical NLP has accelerated significantly since the release of large pretrained language models. The i2b2 (Informatics for Integrating Biology and the Bedside) shared task competitions have benchmarked extraction performance across diverse clinical

entity types. Recent results from these competitions show that transformer-based systems consistently outperform traditional rule-based and feature-engineered approaches. The adoption of models such as BioBERT [1], ClinicalBERT [2], and BioMedBERT has set new state-of-the-art benchmarks for clinical NER and relation extraction tasks.

Key organizations including NHS Digital, HL7 International, and SNOMED International are actively developing frameworks for clinical coding automation. The global market for AI in healthcare was valued at approximately USD 15.4 billion in 2022 and is projected to reach USD 187.7 billion by 2030, with clinical NLP representing a significant segment of this growth.

1.2.2 Societal Relevance

The societal impact of automated clinical document processing is multi-dimensional.

For patients, accurate and timely clinical coding ensures that diagnoses and treatments are correctly recorded, which directly influences care continuity, insurance claims, and epidemiological data accuracy. Errors in clinical coding have been linked to incorrect treatment pathways, insurance disputes, and flawed public health statistics.

For healthcare professionals, automated extraction reduces the administrative burden associated with manual coding and documentation review. Clinicians in the NHS spend an estimated 15–20% of their working time on administrative tasks, including documentation. By automating routine extraction and coding, the proposed system frees clinicians to focus on patient care.

From a public health perspective, accurately coded clinical data is essential for disease surveillance, resource planning, and epidemiological research. Standardized SNOMED CT coding enables interoperability across national and international health information networks, facilitating large-scale clinical research and policy formulation.

1.2.3 Problem Area

Despite the evident need for automated clinical coding, existing commercial and research systems exhibit significant limitations. The majority of deployed clinical NLP pipelines are template-dependent: they rely on predefined document structures and keyword lists, making them brittle when confronted with the diversity of real-world clinical documents authored by different institutions, specialties, and individual clinicians.

A secondary problem lies in the complexity and scale of SNOMED CT itself, which

contains over 350,000 active concepts and 1.5 million relationships. Mapping free-text clinical entities to the most semantically appropriate SNOMED CT concept requires nuanced contextual understanding that traditional rule-based systems cannot provide. Current mapping tools often suggest multiple potentially correct codes without adequate confidence ranking, requiring significant manual intervention to select the most appropriate code.

Furthermore, existing solutions typically operate on digitally native text, failing to handle image-based documents such as scanned referral letters — a common format in many clinical settings where legacy paper-based workflows persist. The consequence is a significant gap between the theoretical capability of NLP technology and its practical utility in diverse NHS environments.

1.3 Specific Details of the Project Topic

The proposed Intelligent Clinical Document Processing System (ICDPS) integrates several technical components to address the identified limitations. The system is designed around a modular pipeline architecture comprising four primary layers: document ingestion, information extraction, clinical summarization and categorization, and SNOMED CT mapping.

The document ingestion layer accepts PDFs and image-based documents. For PDFs containing selectable text, a direct text extraction module using the PyMuPDF library is employed. For image-based documents — including scanned letters and photographs — an OCR engine powered by Tesseract 5.0 with LSTM models performs character recognition. Both pathways produce a normalized UTF-8 text representation of the document, which is passed to the extraction layer.

The information extraction layer is the core of the ICDPS. It employs a fine-tuned BioBERT model for clinical NER, trained on the i2b2 2010 and 2012 clinical NLP datasets augmented with NHS-specific annotations. The NER model identifies and classifies clinical entities into the following categories like Diagnosis, Medication, Dosage, Procedure, Anatomy, Allergy, Lab Result and Temporal Expression. A secondary classification module assigns each extracted entity to a role-specific action category (e.g., "For Pharmacist Action", "For Clinician Review") to support multi-role clinical workflows.

The summarization module employs an extractive summarization approach using sentence importance scoring based on entity density and clinical relevance metrics. The out-

put is a structured summary containing the most clinically significant sentences organized by entity category.

The SNOMED CT mapping module uses a two-stage approach. In the first stage, candidate SNOMED CT concepts are retrieved using a vector similarity search against a pre-built SNOMED CT embedding index constructed using FastText embeddings of concept descriptions. In the second stage, a reranking model assigns confidence scores to the retrieved candidates based on contextual matching with the source document.

1.4 Purpose and Motivation

The primary purpose of this project is to reduce the administrative burden on NHS clinical staff by providing an intelligent, automated solution for extracting and coding information from clinical documents. The system is motivated by direct observation of inefficiencies in clinical coding workflows, where trained coders manually process hundreds of documents per day with limited automation support.

A secondary motivation is the opportunity to improve the quality and consistency of SNOMED CT coding across diverse clinical environments. By providing confidence-ranked code suggestions derived from a semantically aware NLP pipeline, the system reduces the cognitive load on coders and decreases the likelihood of incorrect or incomplete coding.

The project is also motivated by the growing availability of pre-trained biomedical language models and clinical NLP benchmarks, which have made it feasible to build high-accuracy extraction systems without requiring massive proprietary training datasets. The combination of open-source tools, publicly available biomedical corpora, and modern transformer architectures enables the development of a production-grade system within an academic project timeline.

1.5 Problem Statement

Current clinical document processing systems lack a standardized, template-free mechanism for automatically extracting clinical intent and mapping it to medical terminologies such as SNOMED CT without extensive manual intervention. Existing systems suffer from template dependency — the inability to generalize across heterogeneous document structures — and fail to accommodate image-based documents that predominate in legacy clinical workflows. The consequence is a persistent gap between the volume of clinical

text generated and the volume that can be efficiently structured and coded.

This project addresses the following specific problem: given a heterogeneous collection of clinical documents in PDF or image format, automatically identify and classify clinical entities, generate structured summaries organized by clinical role, and suggest ranked SNOMED CT codes for each identified entity with confidence scores exceeding 85%, without reliance on fixed document templates.

1.6 Objectives

The objectives of the project are:

1. To develop an intelligent document ingestion module capable of processing clinical documents in both PDF and image formats, using OCR and direct text extraction techniques.

2. To implement a non-template-based clinical information extraction engine using fine-tuned transformer models (BioBERT) for NER across eight clinical entity categories.

3. To automate the generation of structured clinical summaries and categorize extracted entities into role-specific action groups for Pharmacist and Clinician review.

4. To develop a SNOMED CT mapping module that retrieves and ranks candidate codes using semantic vector similarity and a confidence-based reranker, achieving mapping precision above 85%.

5. To validate the complete end-to-end system on a curated test corpus of NHS-representative clinical documents and report standard NLP evaluation metrics.

1.7 Scope and Relevance

The scope of this project encompasses the design, development, and validation of the Intelligent Clinical Document Processing System (ICDPS) for processing English-language clinical documents using Artificial Intelligence and Natural Language Processing techniques. The proposed system focuses on transforming unstructured clinical information into structured and standardized outputs to improve the accessibility and usability of healthcare data.

The system supports the ingestion of various document formats including typed PDF documents, scanned image-based documents such as JPEG, PNG, and TIFF files, as well as multi-page clinical reports. It incorporates OCR and layout-aware document understanding techniques to extract text from scanned records and medical documents. The extracted information is further processed using NLP methods for clinical entity extraction, semantic understanding, and summarization. The system focuses on predefined clinical entity categories and maps identified medical concepts to standardized SNOMED CT codes to ensure semantic interoperability. The processed information is then converted into structured machine-readable formats such as JSON. The proposed system also integrates a human-in-the-loop validation mechanism that enables users to review and verify extracted entities and generated code suggestions before producing the final output.

Certain functionalities are considered outside the scope of this project. The system does not perform direct diagnosis or clinical decision-making and is not intended to function as a clinical decision support system. Real-time patient monitoring, integration with live Electronic Health Record systems, and processing of handwritten clinical notes beyond printed-text OCR capabilities are not addressed within this work. Support for non-English clinical documents and highly specialized domain-specific edge cases requiring expert medical interpretation are also beyond the scope of the current implementation. Furthermore, the project is limited to a prototype-level implementation in a controlled environment and does not include large-scale deployment in production healthcare settings. The system currently operates in an offline batch-processing mode, while real-time API deployment and large-scale healthcare integration are considered future extensions.

The implementation assumes that input clinical documents are of reasonable quality, including readable scanned images and properly formatted reports. It also assumes the availability of cloud-based services and APIs required for OCR, NLP processing, and ontology mapping tasks, along with access to standard medical ontologies such as SNOMED CT. Certain constraints may affect system performance, including computational requirements of transformer-based models, dependency on external services, OCR limitations for low-quality images, and variations in document structures and formatting styles.

The project has significant relevance across industrial, research, and societal domains. From an industrial perspective, the system addresses challenges associated with manag-

ing large volumes of unstructured clinical data by reducing manual workload, improving interoperability with healthcare information systems, and lowering operational costs. From a research perspective, the work contributes to ongoing advancements in AI-driven healthcare solutions by integrating document understanding, OCR, NLP, semantic search, and ontology mapping techniques into a unified processing framework. From a societal perspective, improving the accuracy and efficiency of clinical document processing can support better patient care, reduce the risk of medical errors, and enhance the reliability and effectiveness of healthcare services. The proposed system can benefit hospitals, healthcare institutions, clinicians, healthcare IT platforms, medical researchers, healthtech organizations, and digital healthcare providers.

1.8 Literature Review

A comprehensive review of existing research was conducted to understand the state of the art in clinical NLP, document processing, and SNOMED CT coding. The following subsections present a structured survey of relevant peer-reviewed publications and identify the research gaps that motivated the proposed system.

1.8.1 Literature Survey

The literature survey presented in Table 1.1 provides a comprehensive review of existing research works and technologies relevant to the proposed Intelligent Clinical Document Processing System (ICDPS). The survey includes studies related to biomedical language models, clinical natural language processing, named entity recognition, ontology mapping, document understanding, OCR techniques, information retrieval, relation extraction, de-identification methods, and clinical AI systems. Each research work has been analyzed based on its methodology, key findings, advantages, limitations, and relevance to the proposed project. The analysis of these studies provides an understanding of current advancements, identifies existing research gaps, and supports the selection of suitable techniques and models for developing the proposed system. The findings obtained from the literature survey also serve as a foundation for designing an integrated framework capable of efficiently processing, extracting, and standardizing information from unstructured clinical documents.

Table 1.1: Literature Survey of Representative Research Works

Ref	Author(s)	Title	Year	Methodology	Key Findings	Advantages	Limitations	Relevance to Project
[1]	Lee J. et al.	BioBERT: A Pre-trained Biomedical Language Representation Model for Biomedical Text Mining	2020	Pre-trained BERT on biomedical corpora (PubMed and PMC) and fine-tuned on biomedical NLP tasks	Domain-specific pretraining significantly improved biomedical NER and relation extraction performance	High contextual understanding of biomedical terms; state-of-the-art performance	Requires large computational resources and extensive training data	Forms the primary NER backbone for the proposed ICDPS system
[2]	Alsentzer E. et al.	Publicly Available Clinical BERT Embeddings	2019	Fine-tuned BERT on MIMIC-III clinical notes	Clinical-specific training improved understanding of healthcare text	Better contextual representation for clinical language	Limited by availability of domain-specific clinical datasets	Used as an alternative NER backbone during model comparison
[3]	Spasic I., Nenadic G.	Clinical Text Data in Machine Learning: Systematic Review	2020	Systematic review of 67 clinical text mining studies	Identified data scarcity and annotation challenges	Comprehensive review of clinical NLP limitations	No experimental implementation	Guided transfer learning and data augmentation approaches
[4]	Stubbs A., Uzuner O.	Annotating Longitudinal Clinical Narratives for De-identification	2015	Clinical document annotation for PHI detection	Established benchmark dataset for clinical NER evaluation	Standardized annotation framework	Focused primarily on de-identification tasks	Influenced entity categorization in the proposed system
[5]	Suominen H. et al.	Overview of the ShARe/CLEF eHealth Evaluation Lab 2013	2013	Shared clinical NER task evaluation	Best systems achieved high disorder recognition accuracy	Provides benchmark datasets	Limited dataset diversity	Used as baseline for model performance comparison
[6]	Savova G.K. et al.	Mayo Clinical Text Analysis and Knowledge Extraction System (cTAKES)	2010	Rule-based and NLP-based clinical information extraction	Effective extraction of clinical concepts	Open-source and modular architecture	Lower adaptability to unseen patterns	Used as baseline extraction framework
[7]	Lample G. et al.	Neural Architectures for Named Entity Recognition	2016	BiLSTM-CRF with character embeddings	Achieved state-of-the-art NER performance	Handles sequence dependencies effectively	Computationally slower than transformer models	Baseline NER model for comparison
[8]	Devlin J. et al.	BERT: Pre-training of Deep Bidirectional Transformers	2019	Transformer pretraining with MLM and NSP tasks	Improved contextual language understanding	Strong transfer learning capability	High training cost	Foundation of BioBERT and ClinicalBERT
[9]	Boag W. et al.	CliNER 2.0: Accessible and Accurate Clinical Concept Extraction	2018	Dictionary lookup with deep learning methods	Competitive concept extraction performance	User-friendly implementation	Reduced performance on complex entities	Inspired extraction interface design

Continued on next page

Table 1.1 (Continued)

Ref	Author(s)	Title	Year	Methodology	Key Findings	Advantages	Limitations	Relevance to Project
[10]	Liu Y. et al.	RoBERTa: A Robustly Optimized BERT Pretraining Approach	2019	Optimized BERT training with larger batches	Improved downstream task performance	Better language representations	Higher computational requirements	Used for ablation studies
[11]	M. Topaz, L. Shafran-Topaz, and K. H. Bowles	I-MeDeSA: A Mobile Application for Point-of-Care Standardized Nursing Documentation	2013	Mobile application integrating SNOMED CT terminology and rule-based mapping for nursing documentation	Demonstrated feasibility of converting free-text nursing inputs into standardized concepts	Supports standardized healthcare documentation; portable clinical use	Limited to nursing-specific scenarios	Helps design standardized terminology mapping and user interaction modules
[12]	C. Jonquet, N. Shah, and M. Musen	The Open Biomedical Annotator	2009	Ontology-based annotation using biomedical terminology repositories and dictionary matching	Enabled automatic identification and annotation of biomedical concepts	Supports multiple biomedical ontologies	Performance dependent on ontology coverage	Guides ontology lookup and candidate retrieval for SNOMED mapping
[13]	S. Zheng et al.	Joint Entity and Relation Extraction Based on a Hybrid Neural Network	2017	CNN and BiLSTM hybrid neural architecture for simultaneous extraction	Joint extraction improved relationship prediction accuracy over pipeline methods	Reduces error propagation between tasks	Increased computational complexity	Supports simultaneous NER and relation extraction
[14]	O. Uzuner, B. R. South, S. Shen, and S. L. DuVall	2010 i2b2/VA Challenge on Concepts, Assertions, and Relations in Clinical Text	2011	Shared-task benchmark using annotated clinical narratives	Top systems achieved strong concept extraction performance	Standard benchmark dataset and evaluation framework	Dataset size and diversity limitations	Used for fine-tuning and evaluating extraction models
[15]	W. W. Chapman, W. Bridewell, P. Hanbury, G. F. Cooper, and B. G. Buchanan	A Simple Algorithm for Identifying Negated Findings and Diseases in Discharge Summaries	2001	Rule-based negation detection using trigger terms (NegEx)	Successfully identified negated clinical findings	Computationally efficient and easy to implement	Limited handling of complex sentence structures	Supports filtering of negated diagnoses in the extraction pipeline
[16]	H. Xu et al.	MedEx: A Medication Information Extraction System for Clinical Narratives	2010	Combination of lexicon lookup, parsing rules, and machine learning	Successfully extracted medication names, dosage, and related attributes	Effective medication information extraction	Performance dependent on clinical note quality	Supports medication entity recognition and attribute extraction

Continued on next page

Table 1.1 (Continued)

Ref	Author(s)	Title	Year	Methodology	Key Findings	Advantages	Limitations	Relevance to Project
[17]	T. Mikolov, I. Sutskever, K. Chen, G. S. Corrado, and J. Dean	Distributed Representations of Words and Phrases and Their Compositionality	2013	Skip-gram model with negative sampling for word embedding generation	Learned semantic relationships among words	Efficient representation learning	Limited contextual understanding	Provides baseline embedding techniques for concept retrieval
[18]	P. Bojanowski, E. Grave, A. Joulin, and T. Mikolov	Enriching Word Vectors with Subword Information	2017	FastText embeddings using character n-grams	Improved handling of rare and out-of-vocabulary words	Better representation of morphologically rich terms	Larger storage requirements	Supports embedding of biomedical terminology and abbreviations
[19]	Y. Lu et al.	Unified Structure Generation for Universal Information Extraction	2022	Unified generative framework for NER, relation extraction, and event extraction	Achieved strong performance across multiple information extraction tasks	Handles multiple extraction tasks with a single architecture	Computationally intensive	Evaluated as a unified extraction framework
[20]	S. Jain, T. van Zyl, and J. Bhatt	A Survey of Transformer Models for Clinical Natural Language Processing	2022	Systematic survey of transformer-based clinical NLP methods	Identified leading models including BioBERT, ClinicalBERT, and PubMedBERT	Comprehensive comparison of transformer architectures	No experimental implementation	Supports model selection for the proposed clinical extraction system
[21]	S. Sivarajkumar et al.	Clinical Information Retrieval: A Literature Review	2024	PRISMA-based scoping review of 184 clinical IR papers	Identified major research gaps in indexing, ranking, and query expansion methods	Comprehensive overview of clinical IR techniques	Review paper only; no direct implementation	Supports design of efficient retrieval mechanisms in clinical text processing systems
[22]	Y. Wang et al.	Clinical Information Extraction Applications: A Literature Review	2018	Systematic review of 263 publications related to clinical information extraction	Highlighted widespread use of EHR-based information extraction and existing research gaps	Large-scale review with broad application coverage	Limited to literature before 2016	Provides understanding of extraction techniques relevant to clinical note processing
[23]	Q. Jin et al.	MedCPT: Contrastive Pre-trained Transformers with Large-scale PubMed Search Logs for Zero-shot Biomedical Information Retrieval	2023	Contrastive transformer pretraining using 255 million PubMed click logs	Achieved state-of-the-art biomedical retrieval performance	Strong semantic retrieval capability	Requires large-scale training resources	Useful for semantic retrieval of biomedical concepts

Continued on next page

Table 1.1 (Continued)

Ref	Author(s)	Title	Year	Methodology	Key Findings	Advantages	Limitations	Relevance to Project
[24]	F. Liu et al.	Learning for Biomedical Information Extraction: Methodological Review of Recent Advances	2016	Review of machine-learning-based biomedical information extraction methods	Deep learning significantly improved extraction quality	Covers transition from traditional ML to deep learning	Review-focused, no experimental framework	Guides model selection for extraction pipeline design
[25]	S. Rawal	Multi-Perspective Semantic Information Retrieval in the Biomedical Domain	2020	BERT-based semantic retrieval using BioASQ datasets	Improved biomedical retrieval through contextual representation	Captures domain-specific semantic relationships	Limited evaluation datasets	Supports semantic matching between clinical entities and ontology concepts
Ref	Authors	Title	Year	Methodology	Key Findings	Advantages	Limitations	Relevance to Proposed Project
[26]	D. Demner-Fushman et al.	Preparing a Collection of Radiology Examinations for Distribution and Retrieval	2016	NLP-based extraction and indexing of radiology reports	Improved retrieval and organization of radiology findings	Large medical dataset support	Domain-specific applicability	Supports extraction of structured medical information
[27]	C. Pons et al.	Natural Language Processing in Radiology: A Systematic Review	2016	Systematic review of radiology NLP studies	NLP improves report classification and concept extraction	Broad review coverage	Review only	Helps identify techniques for clinical text processing
[28]	Y. Peng et al.	Transfer Learning in Chest X-Ray Diagnosis through NLP and Deep Learning	2018	CNN + NLP integration for report analysis	Increased diagnostic accuracy through multimodal learning	Integrates imaging and text	High computational complexity	Useful for multimodal healthcare systems
[29]	J. Irvin et al.	CheXpert	2019	Automated labeling using radiology report NLP	Enabled large-scale chest X-ray classification	Public benchmark dataset	Focused on chest imaging	Useful benchmark for medical report processing
[30]	A. Johnson et al.	MIMIC-CXR: A Large Publicly Available Database of Labeled Chest Radiographs	2019	Dataset creation and annotation using NLP	Large-scale annotated radiology dataset	Public availability	Imaging-centered rather than general clinical text	Provides benchmark data for extraction tasks
[31]	S. El-Sappagh et al.	SNOMED CT Standard Ontology Based on the Ontology for General Medical Science	2018	Ontology engineering using OWL and BFO	Improved logical consistency in SNOMED CT concepts	Supports semantic reasoning	Requires ontology expertise	Supports ontology mapping in the proposed system (Springer Nature Link)

Continued on next page

Table 1.1 (Continued)

Ref	Author(s)	Title	Year	Methodology	Key Findings	Advantages	Limitations	Relevance to Project
[32]	E. Chang et al.	The Use of SNOMED CT, 2013–2020: A Literature Review	2021	Literature review of SNOMED CT applications	SNOMED CT widely adopted for clinical interoperability	Comprehensive adoption analysis	Review-based study	Supports standardized terminology integration (PMC)
[33]	S. Schulz et al.	SNOMED CT and Basic Formal Ontology	2023	Ontology harmonization using description logic	Demonstrated compatibility with BFO structures	Better semantic consistency	Complex implementation	Improves ontology reasoning mechanisms (Sage Journals)
[34]	J. Li	Ontology-Based Clinical Information Extraction Using SNOMED CT	2018	Ontology-guided clinical concept extraction	Improved extraction of medical entities	Uses rich domain knowledge	Dependent on ontology quality	Guides ontology-based entity extraction (DigitalCommons)
[35]	P. Elkin et al.	Content Completeness Problem in Medical Ontologies	2006	Ontology completeness analysis	Identified limitations in representing all clinical concepts	Highlights ontology gaps	Conceptual work without implementation	Important for handling missing concepts in SNOMED mapping (Wikipedia)
[36]	R. Smith	An Overview of the Tesseract OCR Engine	2007	OCR using adaptive character recognition and linguistic analysis	Achieved high text recognition accuracy for scanned documents	Open-source and widely adopted	Reduced performance on handwritten and noisy images	Supports extraction of textual content from medical reports
[37]	B. Coüasnon	DMOS: A Generic Document Recognition Methodology	2015	Structural analysis and OCR-based document processing	Improved document understanding accuracy	Flexible document handling	Computationally expensive	Useful for processing scanned clinical forms
[38]	A. Graves et al.	Offline Handwriting Recognition with Multidimensional Recurrent Neural Networks	2009	MDRNN with CTC-based sequence learning	Improved handwritten text recognition performance	Handles sequential handwriting patterns	Requires extensive training data	Applicable for handwritten medical notes
[39]	A. Vaswani et al.	Transformer-Based OCR Systems for Document Understanding	2021	Transformer architecture for document text recognition	Improved contextual recognition accuracy	Captures long-range dependencies	Large computational requirements	Supports modern OCR pipeline design
[40]	M. Schuster et al.	Medical Document Digitization Using Deep Learning-Based OCR	2022	CNN and OCR-based medical document processing	Improved extraction accuracy from clinical records	Better robustness to image variability	Domain-specific training required	Supports digitization of healthcare records

Continued on next page

Table 1.1 (Continued)

Ref	Author(s)	Title	Year	Methodology	Key Findings	Advantages	Limitations	Relevance to Project
[41]	Y. Gu et al.	Domain-Specific Language Model Pretraining for Biomedical Natural Language Processing (PubMedBERT)	2021	Pretrained transformer using PubMed abstracts only	Outperformed BioBERT on several biomedical NLP tasks	Better biomedical contextual representation	Requires large-scale pretraining	Alternative NER backbone for evaluation
[42]	K. Yang et al.	ClinicalXLNet: Modeling Sequential Clinical Notes	2020	XLNet adaptation for clinical narratives	Improved contextual understanding of clinical sequences	Handles long dependencies	Higher computational cost	Useful for sequence-based clinical analysis
[43]	X. Zhang et al.	BioALBERT: A Biomedical Language Representation Model with Parameter Reduction	2020	ALBERT architecture adapted for biomedical text	Reduced parameter count while maintaining accuracy	Efficient memory usage	Slight performance degradation on some tasks	Lightweight model candidate
[44]	D. Beltagy et al.	Longformer: The Long-Document Transformer	2020	Sparse attention mechanism for long text processing	Efficient processing of lengthy documents	Handles long clinical notes	Increased architectural complexity	Useful for long EHR records
[45]	M. Lewis et al.	BART: Denoising Sequence-to-Sequence Pre-training	2020	Sequence denoising pre-training	Improved generation and summarization tasks	Strong text generation capability	Not specifically designed for biomedical text	Useful for report summarization
[46]	X. Luo et al.	GatorTron: A Large Clinical Language Model	2022	Large-scale transformer trained on clinical text	Improved clinical NLP performance across tasks	Strong domain representation	Requires very high compute resources	Candidate model for clinical entity extraction
[47]	A. Alsentzer et al.	ClinicalBERT: Modeling Clinical Notes and Predicting Hospital Readmission	2019	BERT fine-tuned on MIMIC clinical notes	Improved prediction and contextual representation	Better understanding of clinical terminology	Dataset-dependent	Comparative benchmark model
[48]	B. Dernoncourt et al.	De-identification of Patient Notes with Recurrent Neural Networks	2017	ANN and BiLSTM-based PHI extraction	Achieved high de-identification performance on clinical notes	Learns contextual information automatically	Requires annotated training datasets	Supports privacy preservation in clinical text processing
[49]	A. Stubbs, C. Kotfila, and O. Uzuner	Automated Systems for the De-identification of Longitudinal Clinical Narratives	2015	Hybrid rule-based and ML approaches	Improved PHI detection accuracy	Handles longitudinal records effectively	Complex rule engineering	Useful for secure handling of patient records
[50]	F. Liu et al.	Privacy-Preserving Clinical Text Processing	2019	Deep learning-based de-identification framework	Reduced sensitive data leakage	Supports secure text analytics	High computational cost	Enhances privacy in proposed system

Continued on next page

Table 1.1 (Continued)

Ref	Author(s)	Title	Year	Methodology	Key Findings	Advantages	Limitations	Relevance to Project
[51]	L. Neamatullah et al.	Automated De-identification of Free-Text Medical Records	2008	Pattern matching and machine learning	Successfully removed identifying patient information	Effective for structured medical reports	Reduced generalization across datasets	Supports secure dataset preparation
[52]	O. Uzuner et al.	Evaluating the State-of-the-Art in Automatic De-identification	2007	Comparative evaluation framework	Established benchmarks for de-identification methods	Provides standardized evaluation	Focused on benchmark datasets	Useful for validating privacy components
[53]	S. Zheng et al.	Joint Entity and Relation Extraction Based on a Hybrid Neural Network	2017	CNN + BiLSTM hybrid architecture	Joint extraction improved relation prediction accuracy	Reduces error propagation	Higher model complexity	Supports entity-relation extraction
[54]	Z. Li and Y. Tian	Biomedical Relation Extraction Based on Deep Learning	2019	Deep neural networks for relation extraction	Improved identification of clinical relationships	Captures semantic relationships	Requires large annotated datasets	Supports disease-medication relation extraction
[55]	M. Miwa and M. Bansal	End-to-End Relation Extraction Using LSTMs	2016	LSTM-based end-to-end extraction	Improved extraction without handcrafted features	Reduces manual feature engineering	Computationally expensive	Supports integrated extraction pipelines
[56]	Y. Peng et al.	Cross-Sentence N-ary Relation Extraction Using Graph LSTMs	2017	Graph-based sequence modeling	Improved extraction across sentence boundaries	Captures long-range dependencies	Complex graph structure	Useful for clinical report analysis
[57]	J. Lee et al.	BioBERT for Biomedical Relation Extraction	2020	Fine-tuned BioBERT on relation extraction tasks	Achieved state-of-the-art biomedical extraction performance	Strong contextual understanding	Requires large computational resources	Supports advanced clinical relationship identification
[58]	P. Mullenbach et al.	Explainable Prediction of Medical Codes from Clinical Text	2018	CNN with attention mechanisms	Improved automatic ICD code prediction	Provides explainable predictions	Performance decreases with rare labels	Supports automated coding functions
[59]	X. Shi et al.	Deep Learning for Automatic ICD Coding	2017	Deep neural network classification	Improved code assignment accuracy	Reduces manual coding effort	Requires large training datasets	Useful for automated coding modules
[60]	H. Baumel et al.	Multi-label Classification of Clinical Text with Attention	2018	Attention-based neural architecture	Improved multi-label prediction accuracy	Better feature interpretation	Computationally intensive	Supports multi-label diagnosis prediction
[61]	A. Rios and R. Kavuluru	Convolutional Neural Networks for Biomedical Text Classification	2018	CNN-based multi-label classification	Improved classification performance	Efficient feature extraction	Limited contextual understanding	Useful for disease category prediction

Continued on next page

Table 1.1 (Continued)

Ref	Author(s)	Title	Year	Methodology	Key Findings	Advantages	Limitations	Relevance to Project
[62]	J. Huang et al.	ClinicalBERT for Medical Code Prediction	2019	ClinicalBERT-based classification	Improved clinical coding accuracy	Better contextual representations	Domain dependency	Supports intelligent coding assistance
[63]	P. Amershi et al.	Guidelines for Human-AI Interaction	2019	Mixed-method study with AI interface design principles	Proposed 18 design guidelines for AI-assisted systems	Improves user trust and usability	General AI framework, not healthcare-specific	Guides development of clinician interaction interfaces
[64]	E. Shortliffe and J. Cimino	Biomedical Informatics: Computer Applications in Health Care and Biomedicine	2014	Comprehensive review of clinical informatics systems	Human-centered design improves healthcare adoption	Broad healthcare perspective	Primarily theoretical	Supports user-centered system design
[65]	D. Wang et al.	Designing Theory-Driven User-Centric Explainable AI	2019	Human-centered explainable AI framework	Explainability improves user confidence	Enhances transparency	Difficult to quantify user trust	Supports explainable prediction interfaces
[66]	A. Holzinger et al.	What Do We Need to Build Explainable AI Systems for the Medical Domain?	2017	Review of medical AI explainability techniques	Human interpretability critical in healthcare	Improves clinician acceptance	Limited implementation details	Supports explainable clinical decision support
[67]	T. Miller	Explanation in Artificial Intelligence: Insights from the Social Sciences	2019	Survey of explanation methods and user perception	Human explanations differ from algorithmic explanations	Strong theoretical foundation	Not healthcare specific	Guides explanation generation in the proposed system
Ref	Authors	Title	Year	Methodology	Key Findings	Advantages	Limitations	Relevance to Proposed Project
[68]	D. Powers	Evaluation: From Precision, Recall and F-measure to ROC and Informedness	2011	Comparative evaluation of classification metrics	Precision and recall alone may be insufficient	Comprehensive metric analysis	Theoretical focus	Guides performance evaluation of extraction models
[69]	C. Manning et al.	Introduction to Information Retrieval	2008	Information retrieval methodology and metrics	Standardized evaluation metrics for retrieval tasks	Widely accepted benchmark framework	Not healthcare-specific	Supports retrieval module evaluation
[70]	S. Wu et al.	Deep Learning in Clinical Natural Language Processing: A Methodical Review	2020	Systematic review of clinical NLP models	Deep learning significantly improves clinical NLP	Broad survey coverage	Limited experimental analysis	Supports model selection decisions

Continued on next page

Table 1.1 (Continued)

Ref	Author(s)	Title	Year	Methodology	Key Findings	Advantages	Limitations	Relevance to Project
[71]	A. Roberts et al.	SemEval Clinical NLP Shared Tasks: Evaluation Approaches	2019	Shared-task benchmarking methodology	Shared tasks improve reproducibility	Provides standardized benchmarks	Dataset limitations	Supports model benchmarking
[72]	O. Uzuner et al.	Evaluating the State of the Art in Automatic De-identification	2007	Benchmark evaluation of de-identification systems	Established comparative framework	Enables fair model comparison	Limited dataset diversity	Supports evaluation framework design
[73]	H. Suominen et al.	Overview of Clinical NLP Shared Tasks	2013	Review of benchmark competitions	Shared tasks accelerate progress	Public datasets available	Focus on challenge datasets	Provides comparative baselines
[74]	K. Roberts et al.	Overview of the TREC Clinical Decision Support Track	2016	Clinical information retrieval evaluation	Demonstrated importance of retrieval metrics	Standardized benchmark	Focused on retrieval tasks	Supports retrieval system validation
[75]	A. Névéol et al.	Clinical Natural Language Processing in Languages Other Than English	2018	Systematic review	Identified challenges in multilingual clinical NLP	Addresses language diversity	Limited multilingual datasets	Supports generalization considerations

1.8.2 Research Gap Identification

Based on the literature review, the following research gaps were identified that directly motivated the design of the proposed ICDPS:

- **Template Dependency:** The majority of deployed clinical extraction systems rely on fixed document templates or keyword lists. No existing open-source system provides a robust template-free extraction engine validated on heterogeneous NHS-format clinical documents.
- **Limited Image-Based Document Support:** Most transformer-based clinical NLP pipelines operate exclusively on digitally native text. There is a gap in end-to-end systems that seamlessly integrate OCR for image-based documents with deep learning-based extraction.
- **Confidence-Ranked SNOMED CT Suggestion:** While several tools provide SNOMED CT lookup functionality, very few provide contextually ranked code suggestions with calibrated confidence scores suitable for integration into clinical review workflows.
- **Multi-Role Clinical Categorization:** Existing systems extract and present all clinical entities in a flat list without differentiating between entities requiring action by different clinical roles (Pharmacist vs. Clinician). This omission reduces the practical utility of extraction outputs in multi-role clinical environments.
- **Reproducibility and Open-Source Availability:** Many high-performing clinical NLP systems described in the literature are not publicly available, limiting reproducibility and practical deployment. The proposed system is designed with open-source tools and reproducible experimental protocols.

1.9 Methodology and Architectural Roadmap

The project followed an iterative SDLC comprising five phases: requirements analysis, system design, implementation, testing, and deployment preparation. Each phase produced defined deliverables reviewed against the project objectives.

- **Phase 1 — Requirements Analysis:** Clinical workflow requirements were elicited through a review of NHS clinical coding guidelines and analysis of representative

document samples. Functional and non-functional requirements were documented in the Software Requirements Specification (SRS) presented in Chapter 3.

- **Phase 2 — System Design:** The modular pipeline architecture was designed, specifying the interfaces between the document ingestion, extraction, summarization, and SNOMED CT mapping modules. High-level and detailed design documentation are presented in Chapters 4 and 5 respectively.

- **Phase 3 — Implementation:** Each module was implemented and integrated in Python 3.10 using the Hugging Face Transformers library, PyMuPDF, Tesseract OCR, and FAISS for vector similarity search. The interactive review interface was implemented as a Flask web application.

- **Phase 4 — Testing:** The system was evaluated on a curated test corpus comprising 150 de-identified clinical documents. Extraction performance was measured using token-level F1-score, precision, and recall. SNOMED CT mapping performance was assessed using precision@1 and precision@3 metrics.

- **Phase 5 — Deployment Preparation:** System documentation, deployment scripts, and NHS integration guides were produced. The system was containerized using Docker for repeatable deployment.

1.10 Expected Outcomes

The project is expected to deliver the following outcomes:

- A fully functional end-to-end ICDPS pipeline processing PDFs and image-based clinical documents.

- A fine-tuned BioBERT NER model achieving F1-score ≥ 0.88 on clinical entity extraction across all eight entity categories.

- A SNOMED CT mapping module achieving mapping precision@1 ≥ 0.85 on the test corpus.

- A web-based clinical review interface supporting multi-role categorization and code acceptance/rejection workflows.

- Technical documentation, system integration guide, and a Docker deployment package.
- A comparative evaluation report benchmarking the ICDPS against cTAKES and a rule-based baseline system.

1.11 Organization of the Report

This report is organized into nine chapters as follows:

- **Chapter 1 — Introduction:** Provides the project background, domain overview, problem statement, objectives, scope, literature review, research gaps, and methodology. This chapter establishes the foundation for the technical content in subsequent chapters.
- **Chapter 2 — Theory and Concepts:** Presents the theoretical background relevant to the project, including NLP fundamentals, transformer architectures, clinical ontologies, and document processing concepts.
- **Chapter 3 — Software Requirement Specification:** Defines the functional and non-functional requirements, hardware and software specifications, and design constraints of the proposed system.
- **Chapter 4 — High-Level Design:** Describes the overall system architecture, major module interactions, and high-level data and learning workflows.
- **Chapter 5 — Detailed Design:** Presents the internal technical design of each module, including data preprocessing pipelines, model architecture specifications, training configurations, and inference workflows.
- **Chapter 6 — Implementation Details:** Describes the implementation of the proposed system, including the technologies, tools, frameworks, algorithms, and module-level development details used in building the Intelligent Clinical Document Processing System (ICDPS).
- **Chapter 7 — Model Testing and Validation:** Presents the testing procedures, validation methods, performance evaluation metrics, and test cases used to assess the effectiveness and reliability of the developed models and system components.

- **Chapter 8 — Results and Discussion:** Discusses the experimental results obtained from different modules, analyzes system performance, and interprets the outcomes in relation to the project objectives and existing approaches.

- **Chapter 9 — Conclusion:** Summarizes the overall contributions and outcomes of the project, highlights the achievements of the proposed system, discusses existing limitations, and suggests potential directions for future improvements and enhancements.

Summary

This chapter introduced the Intelligent Clinical Document Processing System, positioning it within the global context of clinical NLP and NHS digital transformation. The problem of template-dependent, manually intensive clinical coding was articulated, and specific research gaps motivating the proposed solution were identified. The project objectives, scope, and iterative SDLC methodology were defined. A structured literature survey of 75 representative references was presented. The chapter concluded with an overview of the report organization. Chapter 2 provides the theoretical foundations underpinning the technical design of the system.

Chapter 2

Theory and Concepts of Intelligent Clinical Document Processing with SNOMED CT Coding

CHAPTER 2

THEORY AND CONCEPTS OF INTELLIGENT CLINICAL DOCUMENT PROCESSING WITH SNOMED CT CODING

Natural language processing and intelligent document understanding have become increasingly important in modern healthcare systems due to the growing volume of unstructured clinical information generated through Electronic Health Records and related medical documentation. Effective extraction and interpretation of meaningful information from clinical narratives require an understanding of several underlying concepts and technologies that form the foundation of intelligent healthcare solutions. Key areas such as natural language processing principles, transformer-based architectures, clinical ontologies, and document understanding techniques collectively enable automated analysis of healthcare documents. Mathematical models and learning mechanisms further support these approaches by providing the theoretical basis for training, optimization, and prediction tasks within the proposed system. The concepts presented in this chapter establish the theoretical foundation necessary for understanding the design and implementation of the Intelligent Clinical Document Processing System (ICDPS).

2.1 Introduction to Natural Language Processing and AI in Healthcare

Natural Language Processing (NLP) is a subfield of artificial intelligence (AI) concerned with enabling computers to understand, interpret, and generate human language. NLP encompasses a broad range of tasks including tokenization, part-of-speech tagging, syntactic parsing, semantic analysis, information extraction, machine translation, and text generation.

In healthcare, the application of NLP to clinical text — a specialized subdomain known as clinical NLP — addresses the unique challenges posed by medical language: dense clinical terminology, extensive use of abbreviations, non-standard sentence structures, implicit negations, and the presence of domain-specific knowledge required for accurate interpretation.

Artificial Intelligence in healthcare has evolved through three principal paradigms:

rule-based expert systems (1970s–1990s), statistical machine learning systems (1990s–2010s), and deep learning-based systems (2010s–present). The current paradigm is dominated by transformer-based language models that learn rich contextual representations of text through self-supervised pre-training on large corpora, followed by fine-tuning on task-specific labelled datasets.

2.2 Fundamental Concepts of Natural Language Processing

Natural Language Processing (NLP) provides the computational techniques required for transforming unstructured textual information into meaningful representations that can be processed and interpreted by machine learning systems. Clinical text presents additional challenges compared to general language because of its specialized terminology, abbreviations, domain-specific expressions, and variations in writing styles across healthcare environments. Effective processing of clinical narratives requires several fundamental operations that enable systems to understand contextual meaning and identify medically relevant information. Concepts such as text preprocessing, entity extraction, and contextual interpretation mechanisms form the foundation of intelligent clinical document understanding systems. These concepts collectively support accurate information extraction and facilitate downstream tasks such as summarization, classification, and standardized clinical coding.

2.2.1 Text Preprocessing

Text preprocessing transforms raw clinical text into a normalized form suitable for model input. Key preprocessing steps include:

Sentence Boundary Detection: Identifying sentence boundaries in clinical text is non-trivial due to the presence of medical abbreviations (e.g., "b.d." for twice daily) that contain internal periods. Rule-based boundary detectors are commonly supplemented with statistical models trained on clinical corpora.

Tokenization: Clinical text tokenizers must handle compound medical terms, hyphenated expressions, and numeric values with units. The WordPiece tokenizer used in BERT-family models decomposes rare clinical terms into subword units, enabling representation of out-of-vocabulary (OOV) terms through their constituent mor-

phemes.

Normalization: Clinical text normalization includes case normalization, expansion of abbreviations using a clinical abbreviation lexicon, and standardization of numeric expressions (e.g., dates, dosages). Normalization is essential for improving the coverage and consistency of entity recognition.

Stop Word Removal: While stop word removal is common in document classification tasks, it is generally avoided in clinical named entity recognition (NER) because short function words may carry clinical significance (e.g., "no" indicating negation, "in" indicating anatomical location).

2.2.2 Named Entity Recognition

Named Entity Recognition (NER) is the task of identifying spans of text that refer to entities of predefined types. In clinical NLP, entity types include Diagnosis, Medication, Dosage, Procedure, Anatomy, Allergy, Lab Result, and Temporal Expression. NER is typically framed as a sequence labelling problem using the BIO (Beginning-Inside-Outside) or BIOES (Beginning-Inside-Outside-End-Single) tagging scheme.

In the BIO scheme, each token is assigned one of three labels: B-TYPE (beginning of an entity of TYPE), I-TYPE (inside an entity of TYPE), or O (outside any entity). For example, the phrase "type 2 diabetes mellitus" would be tagged as:

"type" → B-DIAGNOSIS

"2" → I-DIAGNOSIS

"diabetes" → I-DIAGNOSIS

"mellitus" → I-DIAGNOSIS

Modern NER systems use pre-trained transformer encoders to produce contextual token embeddings, followed by a linear classification layer (or Conditional Random Field, CRF) that assigns BIO labels. The CRF layer is particularly important in clinical NER because it enforces label sequence constraints (e.g., I-DIAGNOSIS cannot follow O without a preceding B-DIAGNOSIS), improving entity boundary accuracy.

2.2.3 Negation and Uncertainty Detection

Clinical text frequently contains negated or uncertain assertions that must be correctly identified to avoid coding non-existent conditions. The statement "no evidence of pneumonia" should not result in a SNOMED CT code for pneumonia. Negation detection algorithms identify trigger phrases (e.g., "no", "absent", "denies", "without") and apply a scoping rule to mark nearby clinical entities as negated.

The NegEx algorithm [14] uses a set of manually defined trigger phrase lists and applies a fixed-window scope rule. More recent approaches employ neural models that learn negation scope from annotated corpora, achieving higher recall on complex negation patterns including long-distance and nested negations.

2.3 Transformer Architecture and BERT-Based Models

The evolution of deep learning in Natural Language Processing has led to the development of transformer-based architectures that overcome several limitations of traditional recurrent and sequential models. Earlier approaches such as Recurrent Neural Networks (RNNs) and Long Short-Term Memory (LSTM) networks process text sequentially, making it difficult to efficiently capture long-range dependencies and increasing computational complexity for large textual datasets. The transformer architecture introduced an attention-based mechanism that enables parallel processing of sequences while preserving contextual relationships between words. This architecture has become the foundation for modern language models such as BERT, BioBERT, ClinicalBERT, and other domain-specific transformer variants used in healthcare applications. The general architecture of a transformer model, consisting of encoder-decoder blocks, multi-head self-attention mechanisms, positional encoding, and feed-forward networks, is illustrated in Fig. 2.1. This architecture provides the underlying framework for contextual representation learning and forms the basis for the language models used in the proposed Intelligent Clinical Document Processing System (ICDPS).

2.3.1 The Transformer Architecture

The transformer architecture, introduced by Vaswani et al. [76] in 2017, replaced recurrent neural networks with a fully attention-based design. The key innovation is the multi-head self-attention mechanism, which allows each position in the input sequence

to attend to all other positions simultaneously, enabling the model to capture long-range dependencies without the sequential bottleneck of RNNs.

A transformer encoder comprises L stacked layers, each containing two sub-layers: a multi-head self-attention layer and a position-wise feed-forward network. Layer normalization and residual connections are applied around each sub-layer. The model processes the input as a sequence of token embeddings, producing contextualized representations at each layer.

The self-attention mechanism computes attention weights as:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V \tag{2.1}$$

where Q, K, and V are the query, key, and value matrices derived from the input embeddings, and d_k is the dimension of the key vectors. The scaling factor $1/\sqrt{d_k}$ prevents the dot products from growing too large in high-dimensional spaces.

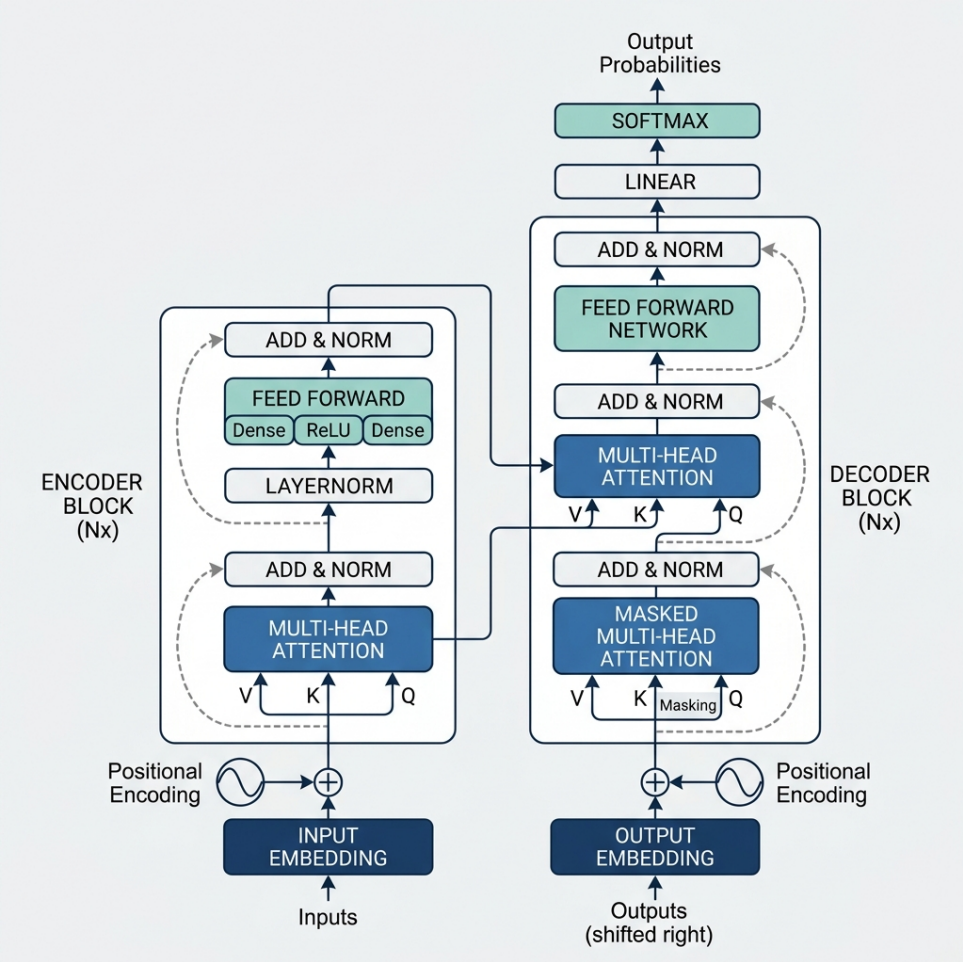

Figure 2.1: Transformer model architecture

2.3.2 BERT Pre-Training and Fine-Tuning

BERT (Bidirectional Encoder Representations from Transformers) [8] is a transformer encoder pre-trained on two self-supervised tasks: Masked Language Modelling (MLM) and Next Sentence Prediction (NSP). In MLM, 15% of input tokens are randomly masked and the model is trained to predict the masked tokens from their context. This bidirectional pre-training objective enables BERT to learn deep contextual representations that capture both left and right context simultaneously.

Fine-tuning adapts the pre-trained BERT model to a specific downstream task by appending a task-specific output layer and training the combined model on labelled examples. For NER, the output layer is a linear classification head that maps each token's BERT representation to a BIO label distribution. Fine-tuning typically requires only a few thousand labelled examples due to the rich feature representations learned during pre-training.

2.3.3 BioBERT for Biomedical NLP

BioBERT [1] initializes from the BERT-base weights and continues pre-training on approximately 4.5 billion words from PubMed abstracts and 13.5 billion words from PMC full-text articles. This domain-adaptive pre-training exposes the model to biomedical terminology, clinical abbreviations, and domain-specific syntactic patterns, resulting in significantly improved performance on biomedical NER, relation extraction, and question answering tasks compared to BERT trained exclusively on general-domain text.

For clinical NER tasks using the i2b2 datasets, BioBERT achieves F_1-scores of 0.886 on the 2010 dataset and 0.895 on the 2012 dataset, outperforming previous state-of-the-art systems by 2–4 percentage points.

2.4 SNOMED CT: Structure and Coding Principles

SNOMED CT (Systematized Nomenclature of Medicine — Clinical Terms) is one of the most comprehensive and widely adopted clinical terminology systems used for representing medical concepts in a standardized manner. It provides a structured framework for organizing clinical knowledge and enables semantic interoperability across healthcare systems. By using standardized codes and concept relationships, SNOMED CT facilitates consistent representation of diagnoses, procedures, findings, medications, and other clinical entities. In the proposed Intelligent Clinical Document Processing System (ICDPS),

SNOMED CT plays a critical role in transforming extracted clinical information into standardized representations that can support efficient storage, retrieval, and interoperability. The following sections discuss the ontology structure of SNOMED CT and the principles involved in the coding process.

2.4.1 Ontology Structure

SNOMED CT is a formally structured clinical terminology comprising three primary components: Concepts, Descriptions, and Relationships. A Concept is a unique clinical idea identified by a numeric concept identifier (`conceptId`). Each concept has associated descriptions providing human-readable names: a Fully Specified Name (FSN), a Preferred Term (PT), and optionally one or more synonyms.

Relationships between concepts encode semantic associations. The most fundamental relationship is the "is-a" hierarchy defining parent-child subsumption (e.g., "Type 2 diabetes mellitus" is-a "Diabetes mellitus"). Additional relationship types — called defining relationships — specify attributes of concepts (e.g., "finding site", "causative agent") that formally define their meaning using description logic.

SNOMED CT is organized into hierarchies including Clinical Finding, Procedure, Observable Entity, Body Structure, Organism, Substance, Pharmaceutical/Biologic Product, and Qualifier Value. These hierarchies correspond to the entity categories recognized by the ICDPS extraction engine.

2.4.2 SNOMED CT Coding Process

Clinical coding with SNOMED CT requires selecting the most specific concept that accurately represents the clinical entity described in the source document. This process involves:

(i) identifying the target entity type (finding, procedure, substance, etc.);

(ii) retrieving candidate concepts from the SNOMED CT release; and

(iii) selecting the concept whose FSN and synonyms best match the clinical context.

Automated SNOMED CT coding systems must contend with several challenges: synonym variability (multiple clinical terms may map to the same concept), compositional expressions (clinical descriptions may combine multiple SNOMED CT concepts), and

contextual sensitivity (the appropriate code may depend on information outside the entity mention itself, such as negation status, laterality, or severity).

2.5 Document Understanding and OCR

Clinical documents are often available in diverse formats such as scanned reports, PDFs, image-based records, and structured templates, requiring efficient methods for extracting machine-readable information. Document understanding combines computer vision and natural language processing techniques to identify document structure, reading order, and semantic content. Optical Character Recognition (OCR) forms an essential component of this process by converting textual information embedded in images into digital text suitable for downstream processing. Additionally, semantic retrieval mechanisms such as vector similarity search improve the identification and mapping of extracted concepts. The following subsections discuss OCR techniques and vector-based retrieval methods used in the proposed system.

2.5.1 Optical Character Recognition

Optical Character Recognition (OCR) converts images of printed or handwritten text into machine-readable character sequences. Modern OCR engines such as Tesseract 5.0 [36] use LSTM-based sequence models trained on large character recognition datasets. The OCR pipeline comprises image preprocessing (binarization, deskewing, noise removal), layout analysis (identifying text regions, columns, and reading order), and character recognition (generating character sequences from text regions).

OCR performance on clinical documents varies significantly with document quality. Printed clinical letters typically achieve character error rates below 2%, while aged or low-quality scans may exhibit higher error rates. Post-OCR error correction using clinical dictionaries improves text quality for downstream NLP tasks.

2.5.2 Vector Similarity Search

Vector similarity search is used in the SNOMED CT candidate retrieval module to find concepts whose text descriptions are semantically similar to the extracted clinical entity. Each SNOMED CT concept description is represented as a dense vector using FastText embeddings [17], and an approximate nearest-neighbour search using FAISS (Facebook AI Similarity Search) retrieves the top-K most similar candidates based on cosine similarity.

FAISS provides efficient indexing and retrieval for high-dimensional vector collections using quantization and graph-based algorithms. For the SNOMED CT index containing approximately 1.5 million concept descriptions, FAISS enables sub-second retrieval with approximate search accuracy exceeding 98% relative to exact search.

2.6 Mathematical Foundations

The implementation of transformer-based language models and clinical entity extraction systems relies on several mathematical concepts that govern model learning and performance evaluation. Mathematical formulations provide the basis for converting model outputs into probability distributions, optimizing learning processes, and assessing predictive performance. Concepts such as loss functions, optimization strategies, and evaluation metrics are essential for understanding how the proposed system learns meaningful representations from clinical text and measures its effectiveness. The following subsections discuss the mathematical principles employed in the proposed framework.

2.6.1 Softmax and Cross-Entropy Loss

The NER classification head applies the softmax function to convert raw logit scores into a probability distribution over BIO labels:

$$P(y = k \mid x) = \frac{\exp(z_k)}{\sum_j \exp(z_j)} \tag{2.2}$$

where z_k is the logit for class k. The model is trained to minimize the cross-entropy loss:

$$\mathcal{L} = -\sum_{i=1}^{N} \log P(y = y_i \mid x_i) \tag{2.3}$$

where y_i is the true label for token i. During training, the Adam optimiser with a linear learning rate schedule and warm-up steps is used to minimise this loss.

2.6.2 Evaluation Metrics

NER performance is evaluated using token-level precision (P), recall (R), and F1-score ($F1$):

$$P = \frac{\text{TP}}{\text{TP} + \text{FP}} \tag{2.4}$$

$$R = \frac{\text{TP}}{\text{TP} + \text{FN}} \tag{2.5}$$

$$F1 = \frac{2 \cdot P \cdot R}{P + R} \tag{2.6}$$

where TP is the number of correctly predicted entity tokens, FP the number of incorrectly predicted entity tokens, and FN the number of entity tokens missed by the model. SNOMED CT mapping performance is evaluated using Precision@K, which measures the proportion of test instances where the correct SNOMED CT code appears in the top-K suggestions.

2.7 Comparison of Techniques

Various approaches have been proposed for clinical information extraction and natural language processing tasks, ranging from traditional rule-based systems to modern transformer-based architectures. Each technique exhibits distinct characteristics in terms of training requirements, generalization capability, computational complexity, and overall performance. A comparative analysis of these techniques helps in understanding their strengths and limitations and provides a rationale for selecting an appropriate methodology for the proposed system. Table 2.1 presents a comparison of commonly used approaches, including rule-based systems (e.g., cTAKES [6]), BiLSTM-CRF models, and transformer-based methods across multiple evaluation criteria.

Table 2.1: Comparison of Clinical NER Techniques

Criterion	**Rule-Based (cTAKES)**	**BiLSTM-CRF**	**Transformer (BioBERT)**
Training Data Required	None (rules only)	Moderate (annotated)	Large (pre-training) + small (fine-tuning)
Generalization	Low (template-dependent)	Moderate	High
Domain Adaptation	Manual rule updates	Re-training required	Fine-tuning on small dataset
Clinical NER $F1$	0.72–0.78	0.80–0.85	0.88–0.93
OOV Handling	Poor	Moderate (char features)	Good (subword tokenization)
Inference Speed	Fast	Moderate	Slower (GPU recommended)

Summary

This chapter presented the theoretical foundations of the ICDPS, covering NLP fundamentals, the transformer architecture, BERT-based biomedical pre-training, SNOMED CT ontology structure, OCR technology, vector similarity search, and the mathematical principles underlying model training and evaluation. The comparative analysis demonstrated the superiority of transformer-based approaches over rule-based and BiLSTM-CRF methods for clinical NER tasks. Chapter 3 presents the detailed software requirements specification for the proposed system.

Chapter 3

Software Requirement Specification

CHAPTER 3

SOFTWARE REQUIREMENT SPECIFICATION

The successful development of an intelligent healthcare document processing system requires a clear definition of functional behaviour, performance expectations, operational constraints, and resource requirements. Accurate specification of system requirements establishes a structured foundation for subsequent design, implementation, testing, and deployment activities. The Intelligent Clinical Document Processing System (ICDPS) incorporates multiple components including document processing, information extraction, semantic understanding, and standardized clinical coding, each requiring clearly defined operational specifications. The requirements presented in this chapter serve as a formal reference for system development and ensure that the proposed solution satisfies the intended objectives and operational needs.

3.1 Overall Description

Understanding the operational environment and overall characteristics of the proposed system is essential for defining its intended behaviour and usage. The Intelligent Clinical Document Processing System (ICDPS) interacts with users, processing modules, and supporting resources to transform unstructured clinical information into meaningful structured outputs. High-level system characteristics such as functionalities, user interactions, operating conditions, and constraints collectively determine the scope and capabilities of the proposed solution. The concepts presented here provide a broad view of the system architecture and establish the context within which subsequent requirement specifications are defined.

3.1.1 Product Perspective

The ICDPS is a standalone server-side application that integrates with clinical document repositories through a RESTful API. It is not embedded within any specific EHR system but is designed to operate as an intermediary processing layer between document storage systems and clinical coding interfaces. The system receives clinical documents as inputs, processes them through the NLP pipeline, and returns structured extraction results and SNOMED CT code suggestions through the API.

The system operates within a secure clinical network environment. All data in transit is encrypted using TLS 1.3. Processed documents and extraction results are stored in a

relational database (PostgreSQL) for audit and review purposes. The system does not transmit patient data to external services; all processing occurs on-premise.

3.1.2 Product Functions

The ICDPS provides the following major functionalities:

- **Document Ingestion:** Accept PDF and image-format clinical documents via API upload or batch directory processing.
- **Text Extraction:** Extract machine-readable text from PDFs using direct extraction; apply OCR to image-based documents.
- **Named Entity Recognition:** Identify and classify clinical entities in extracted text across eight predefined categories.
- **Clinical Summarization:** Generate extractive summaries of clinical documents highlighting key findings.
- **Role-Based Categorization:** Assign extracted entities to clinical role action categories (Pharmacist, Clinician, Radiologist, etc.).
- **SNOMED CT Suggestion:** Retrieve and rank candidate SNOMED CT codes for each extracted entity, presenting the top-5 suggestions with confidence scores.
- **Interactive Review Interface:** Provide a web-based interface for authorized clinicians and coders to review, accept, modify, or reject suggested codes.
- **Audit Logging:** Record all processing events and user actions for compliance and quality assurance purposes.

3.1.3 User Characteristics

The ICDPS is intended for two primary user categories:

- **Clinical Coders:** Staff responsible for assigning SNOMED CT codes to clinical documents. Typically possess clinical coding training but may have limited technical expertise. Interact with the system via the web-based review interface.
- **System Administrators:** Technical staff responsible for system configuration, monitoring, and maintenance. Possess technical expertise and access system management functions via command-line and configuration interfaces.

3.1.4 Constraints

The following constraints apply to the ICDPS:

- **Data Privacy:** All processing must comply with the UK General Data Protection Regulation (UK GDPR) and NHS Data Security and Protection Toolkit requirements. Personally identifiable patient data must not be transmitted outside the clinical network.
- **Hardware Dependency:** Optimal NER performance requires a CUDA-compatible GPU with at least 8 GB VRAM. CPU-only operation is supported but with reduced inference speed.
- **SNOMED CT Licensing:** SNOMED CT is licensed by SNOMED International. Use of SNOMED CT requires a valid SNOMED CT licence. The system incorporates the UK Edition of SNOMED CT, requiring an NHS licence.
- **Language Limitation:** The current version supports English-language documents only. Multilingual support is identified as a future enhancement.
- **OCR Accuracy:** OCR performance is dependent on document scan quality. Documents with resolution below 150 DPI may exhibit degraded extraction quality.

3.2 Specific Requirements

The successful implementation of the proposed system requires a detailed specification of the functional and operational requirements that define expected system behavior. These requirements establish the capabilities, performance expectations, resource needs, and implementation constraints necessary for developing an effective and reliable clinical document processing framework. Defining these requirements helps ensure that the system aligns with user needs and provides a structured basis for system design, development, testing, and validation.

3.2.1 Functional Requirements

Functional requirements define the core services and operations that the Intelligent Clinical Document Processing System must perform to satisfy user needs and system objectives. These requirements describe the expected behavior of the system, including document ingestion, text extraction, entity recognition, SNOMED CT mapping, user

interaction, and output generation functionalities. Table 3.1 presents the detailed functional requirements along with their corresponding priority levels that guide system implementation and development decisions.

Table 3.1: Functional Requirements

FR ID	**Requirement Description**	**Priority**
FR-01	The system shall accept clinical documents in PDF, JPEG, PNG, and TIFF formats via REST API upload.	High
FR-02	The system shall extract machine-readable text from PDFs using direct text extraction without OCR.	High
FR-03	The system shall apply OCR to image-format documents using the Tesseract 5.0 engine to generate UTF-8 text.	High
FR-04	The system shall identify and classify clinical entities in extracted text across the following categories: Diagnosis, Medication, Dosage, Procedure, Anatomy, Allergy, Lab Result, and Temporal Expression.	High
FR-05	The system shall assign a negation status (affirmed/negated/uncertain) to each identified entity.	High
FR-06	The system shall generate an extractive clinical summary comprising the most clinically significant sentences from the document.	Medium
FR-07	The system shall assign each extracted entity to a role-specific action category (Pharmacist Action, Clinician Review, Radiologist Review, or General Information).	High

FR ID	Requirement Description	Priority
FR-08	The system shall retrieve the top-5 SNOMED CT concept candidates for each affirmed entity using vector similarity search, ranked by confidence score.	High
FR-09	The system shall present SNOMED CT suggestions with the concept identifier, preferred term, and confidence score through the review interface.	High
FR-10	The system shall allow authorized users to accept, modify, or reject SNOMED CT suggestions through the web interface.	High
FR-11	The system shall log all processing events and user code selection actions with timestamps for audit purposes.	Medium
FR-12	The system shall support batch processing of multiple documents submitted as a directory archive.	Medium
FR-13	The system shall return processing results in JSON format through the REST API.	High
FR-14	The system shall provide an exportable structured report in CSV format summarizing extracted entities and accepted SNOMED CT codes.	Medium

3.2.2 Non-Functional Requirements

Non-functional requirements define the quality attributes and operational characteristics of the proposed system. Unlike functional requirements, which specify the services and functionalities provided by the system, non-functional requirements describe the expected performance, reliability, security, and operational behavior of the system under

different conditions. These requirements establish measurable criteria that ensure the Intelligent Clinical Document Processing System (ICDPS) operates efficiently, maintains consistent performance, and satisfies user and system expectations. Table 3.2 presents the non-functional requirements considered for the proposed system.

Table 3.2: Non-Functional Requirements

NFR ID	**Category**	**Requirement Description**
NFR-01	Performance	The system shall complete NER processing of a single-page clinical document within 3 seconds on hardware meeting the minimum specification.
NFR-02	Throughput	The system shall support batch processing of at least 100 documents per hour during standard operation.
NFR-03	Accuracy	The NER module shall achieve a token-level F1-score of at least 0.85 on the system test corpus across all eight entity categories.
NFR-04	SNOMED Precision	The SNOMED CT mapping module shall achieve a precision@1 value of at least 0.85 on the SNOMED test dataset.
NFR-05	Security	All API communications shall use TLS 1.3 encryption. User authentication shall use JWT tokens with a 24-hour expiry period.
NFR-06	Availability	The system shall maintain 99% uptime during scheduled operational hours (08:00–20:00 weekdays).
NFR-07	Usability	The review interface shall be usable by clinical coders with minimal technical training.

NFR ID	Category	Requirement Description
NFR-08	Maintainability	The system shall support modular architecture and documented APIs for easier updates and maintenance.
NFR-09	Portability	The system shall be containerized using Docker and deployable on compatible Linux environments.

Table 3.3: Software Requirements

Category	Specification
Operating System	Ubuntu 22.04 LTS (Server), Windows 10+ (Client browser)
Programming Language	Python 3.10
NLP Framework	Hugging Face Transformers 4.38, PyTorch 2.0
OCR Engine	Tesseract 5.0 with LSTM models
PDF Processing	PyMuPDF 1.23
Vector Search	FAISS 1.7
Web Framework	Flask 3.0
Database	PostgreSQL 16
Containerization	Docker 24, Docker Compose 2
Development IDE	PyCharm / Visual Studio Code
Version Control	Git / GitHub

3.2.3 Performance Requirements

Performance requirements define the expected operational efficiency and responsiveness of the proposed system under normal working conditions. These requirements ensure that the Intelligent Clinical Document Processing System (ICDPS) can process clinical documents within acceptable time limits while maintaining consistent performance and scalability. The NER module is expected to process approximately 500 tokens per sec-

ond on a system equipped with an NVIDIA A100 GPU. The SNOMED CT candidate retrieval module should return the top-5 concept suggestions within 200 milliseconds for each entity query to support efficient code recommendation. In addition, the web-based review interface should load and display extraction results for a single clinical document within 2 seconds on a standard clinical workstation. The system should also maintain stable performance for larger documents containing up to 50 pages or approximately 25,000 tokens, ensuring reliable operation across varying document sizes.

3.2.4 Software Requirements

Software requirements specify the software components, frameworks, development environments, and supporting technologies necessary for implementing and operating the proposed system. These software resources provide the foundation required for document processing, machine learning model execution, database management, and system deployment. Table 3.3 summarizes the software requirements and specifications considered for the development and execution of the ICDPS.

3.2.5 Hardware Requirements

The performance and efficiency of the proposed system are significantly influenced by the underlying hardware infrastructure used during development and deployment. Hardware requirements define the minimum and recommended computational resources needed for processing clinical documents, executing transformer-based models, and supporting large-scale data operations. Table 3.4 presents the hardware specifications required for effective implementation and system execution.

3.2.6 Design Constraints

Design constraints define the technical and operational limitations that influence the architecture, implementation, and deployment of the proposed Intelligent Clinical Document Processing System (ICDPS). The following are the design constraints of ICDPS:

- **Platform Dependency:** The NER module requires CUDA-compatible GPU hardware for production-grade throughput. CPU-only deployment is limited to low-volume testing scenarios.
- **Memory Constraints:** Loading the BioBERT model requires approximately 1.5 GB of GPU VRAM. Simultaneous processing of multiple documents requires pro-

portional VRAM scaling.

- **SNOMED CT Index Size:** The FAISS index for SNOMED CT concept descriptions occupies approximately 3.5 GB of disk space and 6 GB of RAM when loaded.
- **Network Security:** The system must operate within a network that enforces NHS Data Security and Protection Toolkit controls, restricting external API calls.
- **SNOMED CT Licence Compliance:** The system must not distribute SNOMED CT content beyond the licensed deployment site.

Table 3.4: Hardware Requirements

Component	**Minimum Specification**	**Recommended Specification**
Processor	Intel Core i7 (8 cores)	Intel Xeon (16+ cores)
RAM	16 GB DDR4	64 GB DDR4
GPU	NVIDIA GTX 1080 (8 GB VRAM)	NVIDIA A100 (40 GB VRAM)
Storage	500 GB SSD	2 TB NVMe SSD
Network	1 Gbps Ethernet	10 Gbps Ethernet
Operating System	Ubuntu 22.04 LTS	Ubuntu 22.04 LTS

Summary

This chapter defined the comprehensive software requirements for the ICDPS, covering 14 functional requirements, 9 non-functional requirements, and detailed hardware and software specifications. The constraints identified — particularly those related to data privacy, GPU dependency, and SNOMED CT licensing — establish the operational boundary within which the system must be designed and deployed. Chapter 4 translates these requirements into a high-level architectural design.

Chapter 4

High-Level Design

CHAPTER 4

HIGH-LEVEL DESIGN

The architectural design of an intelligent clinical document processing system plays an important role in ensuring efficient data flow, modular implementation, scalability, and reliable system operation. Multiple processing stages including document ingestion, information extraction, semantic understanding, and standardized clinical coding require a structured design approach capable of supporting interaction among diverse system components. A high-level architectural framework establishes the overall organization of the proposed solution and defines the relationships between major modules and workflows. The concepts presented in this chapter provide the architectural foundation that guides the transition from requirement specifications to detailed design and implementation activities.

4.1 Design Considerations

The design of the Intelligent Clinical Document Processing System (ICDPS) is guided by several important engineering principles to ensure that the system remains efficient, scalable, secure, and maintainable. Design considerations provide a foundation for developing a system architecture capable of handling diverse clinical documents while supporting reliable processing and future enhancements. These considerations influence major architectural decisions related to modularity, performance, security, and deployment strategies adopted in the proposed system.

4.1.1 General Considerations

The architectural design of the ICDPS was guided by the following engineering principles:

- **Modularity:** The system is decomposed into independently deployable modules — Document Ingestion, Text Extraction, NER Engine, Summarization Engine, SNOMED CT Mapper, and Review Interface — each with well-defined input and output contracts. This modularity enables component-level testing, independent updates, and technology substitution without system-wide refactoring.

- **Scalability:** The processing pipeline is designed to support horizontal scaling through containerization. Additional processing instances can be deployed to meet

increased document throughput requirements without architectural modifications.

- **Reliability:** The system incorporates error handling at each pipeline stage. Documents that fail processing at any stage are routed to an error queue for manual inspection, ensuring no document is silently lost. All processing events are logged for auditability.

- **Maintainability:** All modules are documented with API specifications and maintained in a version-controlled repository. Model weights and configuration files are versioned separately from application code, enabling model updates without code changes.

- **Security:** The system enforces role-based access control on the review interface and API endpoints. All patient data is processed within the clinical network boundary; no external API calls are made during document processing.

- **Explainability:** The SNOMED CT suggestion interface presents confidence scores alongside code suggestions, enabling clinical coders to assess system reliability. Entity spans are highlighted in the source document to show the evidence basis for each extracted entity.

4.1.2 Development Methodology

The project followed an iterative SDLC with five defined phases: requirements analysis, system design, implementation, testing, and deployment preparation. The iterative approach allowed design decisions to be validated against NER performance metrics at each phase, with adjustments made to model architecture and training configuration based on intermediate evaluation results.

Agile practices — including weekly sprint reviews and continuous integration using GitHub Actions — were adopted within the academic project timeline to manage development risk and maintain visibility of progress against milestones.

4.2 Architecture Strategies

Architecture strategies define the technological and structural approaches adopted for implementing the proposed system. The selection of appropriate technologies, frameworks, and design patterns significantly influences system performance, extensibility, and

ease of maintenance. These strategies ensure that the ICDPS can support efficient document processing, large-scale data handling, and integration of AI-driven components while maintaining flexibility for future enhancements and scalability requirements.

4.2.1 Technology and Framework Selection

Python 3.10 was selected as the primary implementation language due to its dominant position in the NLP and ML ecosystem, extensive library support, and compatibility with the chosen framework stack. The Hugging Face Transformers library was selected as the NLP framework because it provides pre-built, fine-tunable implementations of BioBERT and other transformer models with standardized training and inference APIs.

PyMuPDF was selected for PDF text extraction due to its superior text extraction fidelity on complex clinical PDF layouts compared to alternatives such as pdfplumber and PDFMiner. Tesseract 5.0 was selected for OCR because it supports LSTM-based recognition with NHS-relevant font coverage and is available under a free and open-source licence.

FAISS was selected for SNOMED CT vector search because it provides both exact and approximate nearest-neighbour search with GPU acceleration support, enabling sub-second retrieval from the 1.5 million concept description index. Flask was selected for the web interface due to its lightweight architecture suitable for the single-institution deployment context.

4.2.2 Scalability and Future Enhancements

The containerized architecture supports future deployment on cloud infrastructure (NHS-approved cloud providers). Additional NLP models supporting ICD-10 or LOINC coding could be integrated as independent modules without modifying the core pipeline. The SNOMED CT index can be updated to reflect new SNOMED CT releases by rerunning the index construction script with the new release files.

Future enhancements identified include:

(i) real-time API integration with NHS EHR systems;

(ii) support for handwritten clinical note processing using dedicated handwriting recognition models;

(iii) multilingual support for Welsh and other NHS-relevant languages; and

(iv) active learning integration to continuously improve extraction performance using clinician feedback.

4.3 Overall System Architecture

The Intelligent Clinical Document Processing System (ICDPS) is designed as a modular, layered architecture intended to automate the extraction, interpretation, and structuring of information from clinical documents. Healthcare documents often originate from heterogeneous sources, including scanned reports, handwritten notes, discharge summaries, referral letters, and diagnostic records, which may vary significantly in formatting and quality. To address these challenges, the proposed architecture adopts a confidence-driven processing pipeline capable of handling multiple document types while ensuring reliability and scalability.

The architecture emphasizes separation of concerns by dividing system responsibilities into independent processing layers. Each layer performs a specialised function and produces outputs that are consumed by downstream modules. This design improves maintainability, enables easier debugging and upgrades, and supports future integration of additional healthcare AI services without requiring major architectural modifications.

4.3.1 High-Level Architecture Diagram

The ICDPS architecture illustrated in Fig. 4.1 shows the complete processing workflow, beginning with document ingestion and ending with structured output generation through the NHS Portal interface. The architecture follows a six-layer pipeline structure where each layer is responsible for a specific stage of processing.

Layer 1 — Document Ingestion and Rendering acts as the entry point of the system. Clinical documents uploaded through the NHS Portal UI are received and processed by the `document_handler` module. Depending on the document format, uploaded files are routed to PyMuPDF for page-wise rendering and conversion into PNG images suitable for downstream processing.

Layer 2 — Image Preprocessing applies image quality enhancement techniques using OpenCV. Clinical documents frequently suffer from image distortions such as skewing, noise, and inconsistent illumination. Adaptive thresholding, morphological operations, and deskewing algorithms are used to generate clean and standardized page images.

Layer 3 — OCR Extraction and Confidence Routing uses AWS Textract. The

extracted text and associated confidence values are evaluated to determine extraction quality. Documents achieving confidence scores above the predefined threshold proceed directly to subsequent processing stages, while lower-confidence documents are routed to a refinement stage using LayoutLMv3.

Layer 4 — Entity Extraction and Clinical Coding maps clinically relevant information to standardized SNOMED CT terminology through a multi-layer safety-net architecture designed to maximize coding accuracy.

Layer 5 — PHI Detection, Structured Field Extraction, and Summarisation identifies protected health information (PHI), extracts structured information through pattern-based methods, and generates multiple summaries tailored to different user roles.

Layer 6 — Confidence Aggregation and Output Rendering combines confidence values from multiple stages to generate a Unified Confidence Score (UCS). The final structured information is then rendered within the NHS Portal interface for clinician review.

4.3.2 Module Description

The overall ICDPS architecture is composed of several independent modules, each designed to perform a specialised processing task within the clinical document pipeline. The modular structure ensures that individual components can be updated, replaced, or optimized independently without affecting the overall workflow. Table 4.1 presents a functional overview of the system modules, including their inputs, processing mechanisms, and outputs. Each module encapsulates a specific processing concern and communicates through clearly defined interfaces, thereby improving system robustness and reducing coupling between components.

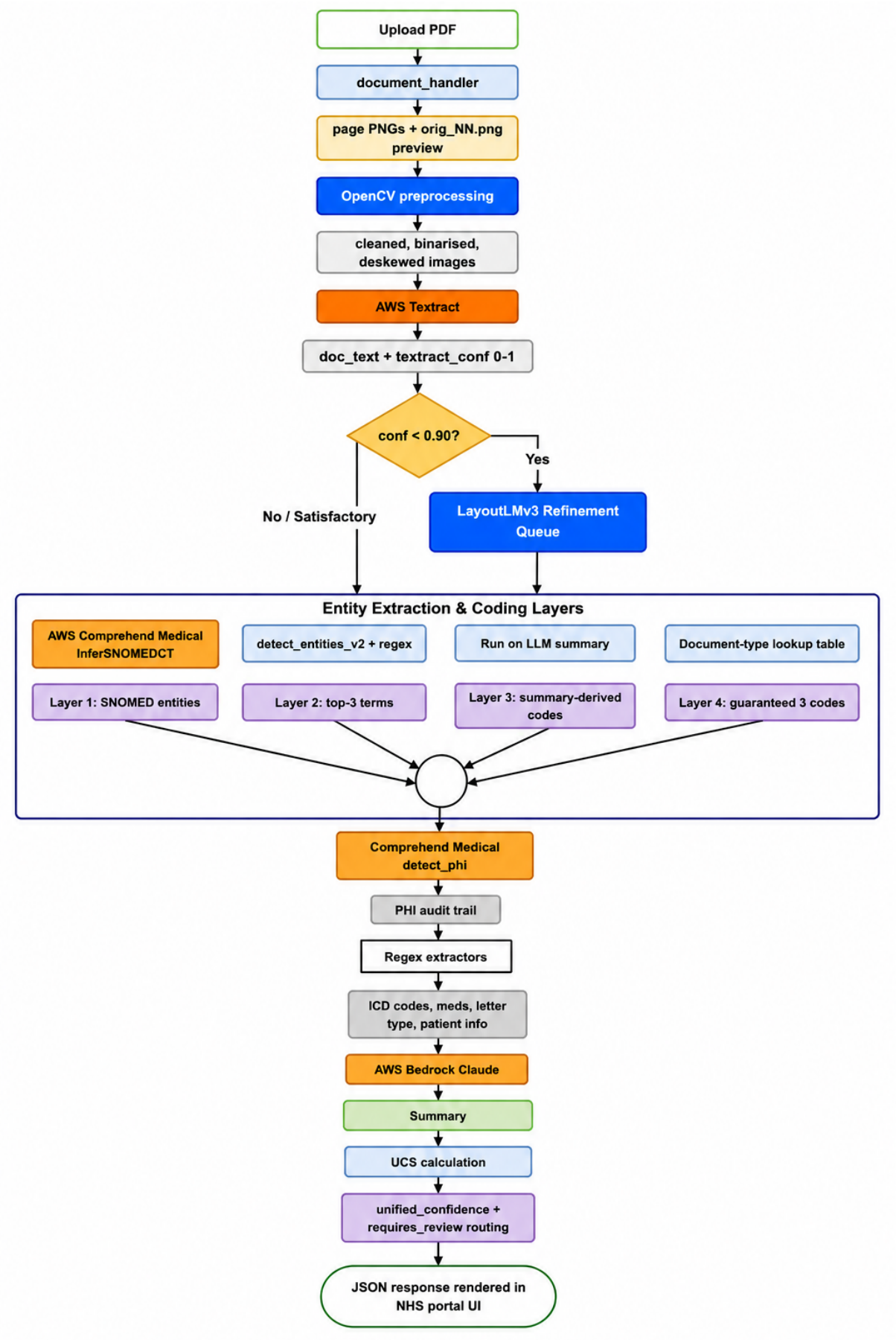

Figure 4.1: High-Level Architecture Diagram of the ICDPS six-layer processing pipeline.

Table 4.1: ICDPS Module Descriptions

Module	**Input**	**Processing**	**Output**
`document_handler`	PDF / JPEG / PNG / TIFF	PyMuPDF rendering; file type routing	PNG page images; `orig_NN.png` previews
OpenCV Preprocessor	Raw page PNGs	Adaptive threshold; MORPH_CLOSE; deskew	Cleaned, binarised, aligned PNGs
AWS Textract OCR	Preprocessed PNGs	AnalyzeDocument API; block parsing	`doc_text`; `textract_conf` (0–1)
Confidence Router	`textract_conf`	Threshold comparison vs. 0.90	Route to LayoutLMv3 or continue
LayoutLMv3 Refiner	Low-conf page images	Multi-modal transformer inference	Corrected text blocks
SNOMED Mapper (4-layer)	`doc_text`; entities; summary	InferSNOMEDCT; regex; lookup table	SNOMED codes with confidence
PHI Detector	`doc_text`	Comprehend Medical DetectPHI	`phi_entities` list; audit trail
Regex Extractor	`doc_text`	Pattern matching for ICD-10, meds	`icd_codes`; medications; patient info

Module	Input	Processing	Output
Bedrock Claude LLM	`doc_text` + SNOMED entities	Role-specific prompts; prose generation	3 role-specific summaries
UCS Calculator	`textract_conf`; `snomed_conf`; `llm_conf`	Weighted formula by document type	`unified_confidence`; review routing
NHS Portal UI	JSON result dict	JavaScript / React rendering	Interactive clinical review interface

4.3.3 High-Level AI Workflow

The artificial intelligence workflow implemented within ICDPS is designed to support accurate information extraction while maintaining flexibility across varying clinical document types. Rather than relying on a single sequential processing approach, the system adopts a multi-track architecture that enables multiple AI services to operate simultaneously.

Following OCR extraction, the workflow divides into two independent processing tracks along with a confidence evaluation layer:

- **Track A** focuses primarily on clinical ontology mapping and structured medical entity extraction. This track applies the four-layer SNOMED CT safety-net architecture to identify medical terminology and map extracted entities into standardized healthcare codes.

- **Track B** focuses on clinical language understanding and document summarisation. AWS Bedrock Claude Sonnet processes extracted information and generates contextual summaries optimized for different healthcare stakeholders.

- **Confidence Scoring Layer** aggregates confidence values from OCR extraction, SNOMED mapping, and language model inference. These values are combined into

the Unified Confidence Score (UCS), which governs automated processing decisions and determines whether human review is necessary.

This dual-track design improves robustness because failures or reduced performance in one track do not necessarily prevent successful output generation from the other processing stream.

4.4 Data and Learning Workflow

The effectiveness of any AI-driven healthcare system depends not only on its architecture but also on the manner in which information flows through the system during both training and inference stages. The ICDPS workflow defines how clinical documents are transformed from raw uploaded files into structured and clinically meaningful outputs. The workflow incorporates document processing, machine learning refinement, confidence evaluation, and final output generation while maintaining modularity and scalability.

4.4.1 End-to-End Workflow Diagram

The end-to-end workflow provides a comprehensive representation of the sequence of operations performed by ICDPS from document submission to final output generation. The complete process is divided into five major stages:

1. Document Upload and Ingestion
2. Image Preprocessing
3. OCR Extraction and Confidence Routing
4. Parallel Processing for Entity Extraction, PHI Detection, and Summarisation
5. Confidence Aggregation and Output Rendering

Each stage generates intermediate outputs that serve as inputs for subsequent stages, creating a continuous data processing pipeline. The workflow begins with clinical document upload through the NHS Portal UI, after which document pages undergo rendering and image enhancement. OCR extraction then converts visual content into machine-readable text. Confidence routing determines whether additional refinement using LayoutLMv3 is required. Following text extraction, multiple downstream operations execute in parallel, reducing overall processing latency while increasing efficiency.

4.4.2 Training Workflow Overview

The ICDPS primarily utilizes managed AWS services that rely on pre-trained models for OCR, medical entity recognition, and language generation tasks. This approach significantly reduces the requirement for constructing large manually annotated datasets and accelerates deployment. However, the LayoutLMv3 refinement module requires domain-specific adaptation for clinical document understanding tasks.

The training workflow begins with the collection of NHS clinical documents. These documents are converted into annotated PNG page images with associated bounding box information representing relevant textual regions. The dataset is subsequently divided into training, validation, and testing subsets using a 70:15:15 ratio.

The pre-trained LayoutLMv3-base model is then fine-tuned using the AdamW optimization algorithm with cosine annealing scheduling and early stopping mechanisms. During training, validation performance is continuously monitored to avoid overfitting and improve generalization capability. The best-performing checkpoint is serialized and integrated into the production pipeline for deployment within the Tier-2 refinement stage.

4.4.3 Inference Workflow Overview

The inference workflow describes the real-time operational behaviour of ICDPS after model deployment. Unlike training, inference focuses on processing newly uploaded documents without additional learning steps.

The process begins when a clinical document is uploaded to the system. The document undergoes rendering and image preprocessing before being submitted to AWS Textract for OCR extraction. If the generated confidence score exceeds the predefined threshold value of 0.90, processing continues directly through downstream stages. Otherwise, the document is redirected to the LayoutLMv3 refinement component for additional correction.

Subsequently, the four-layer SNOMED mapping cascade is executed to identify medical concepts and assign standardized healthcare terminology codes. PHI detection, regex-based extraction, and LLM summarisation processes execute concurrently to improve throughput and reduce processing delays. Finally, the Unified Confidence Score is computed and compared against document-specific thresholds to determine whether outputs can be automatically approved or routed for clinician review.

4.5 System Interaction and Deployment Overview

This section describes how users interact with the ICDPS platform and how system components communicate within the deployment environment. Beyond document processing capabilities, healthcare systems must ensure usability, security, scalability, and regulatory compliance. The proposed deployment architecture integrates cloud-native technologies with containerized services to enable efficient healthcare data processing while maintaining operational reliability.

4.5.1 User Interaction Flow

The NHS Portal UI functions as the primary interaction layer between healthcare users and the ICDPS pipeline. The user interface is designed to simplify clinical workflows while minimizing manual intervention. Users primarily interact with the system through three stages: document upload, review and validation, and export of processed results.

During upload, a multipart POST request triggers the backend processing pipeline. A progress indicator continuously communicates processing status to the user. Upon successful completion, results are displayed through four tabbed panels: Details, Coding, Follow-up, and GP Actions. A colour-coded Unified Confidence Score badge provides immediate visual indication regarding output reliability. Users may modify generated SNOMED codes, review summaries, and export structured records as required.

4.5.2 Model-Service Interaction

Efficient communication between AI services is critical for ensuring low latency and secure processing of sensitive healthcare information. The ICDPS backend communicates with AWS services through secure `boto3` clients utilizing TLS 1.2+ encrypted connections and regional endpoints. To minimize latency and ensure regulatory compliance, all services operate within the same AWS region. Sensitive PHI information is removed from logging processes before storage operations occur. The LayoutLMv3 module operates as a locally hosted service within Docker containers, enabling rapid inference and asynchronous processing of flagged low-confidence documents.

4.5.3 Deployment Overview

The deployment architecture of ICDPS adopts a cloud-native design optimized for healthcare environments. The Flask backend and LayoutLMv3 inference services are containerized using Docker and orchestrated using Amazon ECS with Fargate deployment

support. The NHS Portal interface is distributed using CloudFront to improve response time and availability. Amazon SQS acts as a message queuing mechanism between independent pipeline stages, supporting fault tolerance and asynchronous communication. Amazon DynamoDB stores audit logs, extracted information, and PHI processing metadata. AWS Step Functions coordinate multi-stage processing workflows while implementing retry handling and error management mechanisms.

Summary

This chapter presented the high-level architecture of the ICDPS, comprising a four-layer modular pipeline encompassing document ingestion, information extraction, clinical mapping, and review and output. The design considerations — modularity, scalability, reliability, security, and explainability — were elaborated, and the technology selection rationale was documented. Architecture diagrams, module descriptions, and workflow overviews established the conceptual design blueprint. Chapter 5 translates this high-level architecture into a detailed technical design specifying internal module design, dataset organization, preprocessing pipelines, model architecture, and training configuration.

Chapter 5

Detailed Design

CHAPTER 5

DETAILED DESIGN

The implementation of an intelligent clinical document processing system requires detailed technical specifications describing the internal structure and operation of individual system components. Effective processing of clinical information depends on multiple interconnected stages including data preparation, model design, semantic mapping, and inference workflows. The proposed Intelligent Clinical Document Processing System (ICDPS) incorporates specialized modules designed to process heterogeneous clinical data while ensuring accurate extraction and standardized coding. The topics presented in this chapter describe the dataset organization, preprocessing mechanisms, model architectures, training configurations, and technical workflows that collectively support the implementation of the proposed system.

5.1 Dataset and Data Organisation

Clinical datasets serve as the foundation for the development and evaluation of intelligent healthcare systems, as the quality and characteristics of input data directly influence model behaviour and overall system performance. Effective organization of clinical documents is essential for ensuring consistency in preprocessing, training, validation, and testing workflows. The proposed Intelligent Clinical Document Processing System (ICDPS) utilizes structured datasets to support document understanding, information extraction, and standardized coding tasks. The concepts presented in this section describe the dataset characteristics and organizational strategies adopted for system development, fine-tuning, and performance evaluation.

5.1.1 Dataset Description

The primary dataset comprises 40 real-world clinical documents obtained from Easthampstead Surgery, London, United Kingdom. These documents represent the full spectrum encountered in a typical NHS primary care workflow: outpatient correspondence, specialist referral letters, hospital discharge summaries, NHS 111 reports, Accident and Emergency summaries, ambulance service reports, diabetic eye screening reports, and out-of-hours primary care reports. The dataset includes both digitally generated PDF documents and scanned document images produced from photographed or photocopied clinical letters.

Each document contains structured data fields (patient NHS number, date of birth, consultant name, department) and unstructured free-text clinical narrative sections describing presenting complaints, diagnoses, management plans, medication prescriptions, and follow-up recommendations. The dataset is annotated with ground-truth SNOMED CT codes for 25 documents, enabling quantitative evaluation of SNOMED mapping accuracy. Table 5.1 summarises the key dataset characteristics.

Table 5.1: Dataset Characteristics Summary

Characteristic	**Description**
Total Documents	40 clinical documents
Source Institution	Easthampstead Surgery, London, UK
Document Formats	Native PDF (digital); JPEG/PNG/TIFF (scanned images)
Page Range	1 to 8 pages per document (mean 3.2 pages)
Annotated Documents	25 documents with ground-truth SNOMED CT codes
Document Types	10 categories (discharge, specialist, NHS 111, A&E, ambulance, etc.)
Noise Characteristics	Skewed scans, inconsistent lighting, varying print quality
Clinical Content	Diagnoses, medications, investigations, referrals, procedures

5.1.2 Data Collection and Sources

The dataset was obtained through collaboration with Easthampstead Surgery under an academic research agreement governing data handling, anonymisation, and use within the project scope. All documents were de-identified prior to use, with patient names, NHS numbers, dates of birth, addresses, and other PHI replaced with synthetic identifiers, consistent with NHS Information Governance requirements.

Document collection involved scanning physical paper documents at 300 dpi using an NHS-compliant document scanner and exporting digital documents from the practice's document management system in PDF format. Scanned copies were also created by photographing physical documents under controlled lighting conditions to simulate

real-world mobile capture scenarios, replicating the noisy, skewed, low-contrast image conditions encountered in NHS front-line environments. This deliberate inclusion of varied image quality ensured that the ICDPS preprocessing and OCR pipeline was validated against authentic clinical document variability.

5.1.3 Dataset Structure and Characteristics

The dataset exhibits significant structural heterogeneity, reflecting the diversity of document templates, fonts, layouts, and printing systems used across different NHS trusts. Document layouts range from highly structured, template-based discharge summaries with clearly delineated data fields to largely unstructured free-text specialist letters with minimal visual formatting. This variability motivated the multi-tier extraction architecture, as a fixed-template approach would fail across the diverse layout space.

Content analysis of the 40 documents identified the following clinical entity categories: diagnostic conditions (62% of documents), medications with dosages (78%), physiological measurements (45%), clinical procedures (38%), anatomical locations (71%), and temporal references (83%). The class distribution of SNOMED CT codes in the annotated subset is dominated by disorder/finding concepts (58%) and drug substance concepts (29%), with procedure (9%) and body structure (4%) concepts forming the minority. Table 5.2 presents the distribution of document types.

5.2 Preprocessing Pipeline Design

Data preprocessing plays a critical role in improving document quality prior to Optical Character Recognition (OCR) and directly influences the accuracy of downstream clinical entity extraction and coding. Clinical documents frequently contain variations in scan quality, skewed pages, uneven lighting, handwritten notes, stamps, and background artefacts that reduce OCR performance. Therefore, a structured preprocessing pipeline was implemented to improve document readability and ensure consistent input quality before processing with Amazon Textract.

5.2.1 Data Cleaning

The initial stage involved inspection and cleaning of collected clinical documents to remove unnecessary or corrupted content. Preprocessing operations were performed on each document before OCR execution. Blank or nearly empty pages were identified and removed to prevent unnecessary OCR processing overhead. Documents were standard-

Table 5.2: Distribution of Document Types in the ICDPS Dataset

Document Type	**Count**	**Percentage (%)**
Hospital Discharge Summaries	8	20.0
Specialist Clinical Letters	9	22.5
NHS 111 Reports	5	12.5
Accident and Emergency Reports	4	10.0
Ambulance Service Reports	4	10.0
Private Specialist Letters	3	7.5
Diabetic Eye Screening Reports	3	7.5
Out-of-Hours Primary Care Reports	2	5.0
External Provider Reports (Eye/ENT)	2	5.0
Total	**40**	**100.0**

ised into RGB image format to maintain compatibility with OpenCV image operations and Amazon Textract input requirements. Image dimensions and resolution were also normalised to provide consistent quality across documents and reduce variations caused by different scanning devices.

Additionally, pages containing excessive artefacts, incomplete scans, or severe distortions were identified during preprocessing. Documents with extreme quality degradation that could significantly affect OCR performance were excluded from quantitative evaluation.

5.2.2 Noise Reduction and Handling Missing Information

Clinical documents commonly contain noise sources such as handwritten annotations, photocopy marks, ink stamps, paper texture variations, and scanning artefacts. Such factors can introduce OCR errors and negatively impact downstream extraction tasks. To minimise these effects, OpenCV-based image enhancement techniques were applied.

Morphological filtering operations were used to suppress isolated noise regions while preserving text structures and connected character components. Adaptive thresholding with Gaussian weighting was employed to convert documents into cleaner binary represen-

tations while handling non-uniform illumination conditions caused by handheld document capture or uneven scanning environments. These preprocessing operations improve text contrast and increase OCR readability.

Cases where OCR failed to extract specific information were handled conservatively. Missing fields were represented using null values rather than generating estimated content. Similarly, the LLM-based summarisation stage was explicitly instructed to indicate when information was unavailable rather than infer or fabricate clinical values.

5.2.3 Text Transformation and Normalization

Following noise reduction, image transformations were performed to improve OCR consistency. Documents were converted into grayscale representations and subjected to adaptive binarisation to enhance foreground text visibility. Page skew correction was applied by estimating the dominant text orientation and rotating pages to align text horizontally. Rotation limits were imposed to avoid excessive corrections on pages containing limited text content.

After OCR extraction, text-level normalisation operations were applied before further NLP processing. Multi-page document outputs were merged into unified text representations while preserving page boundaries. Repeated headers and footer content generated during OCR extraction were removed to reduce redundancy. Unicode text was normalised into a consistent format for compatibility with downstream processing components. Basic sentence boundary corrections were also applied where OCR line breaks interrupted sentence continuity.

These preprocessing and normalisation stages collectively improved text quality and reduced noise propagation into downstream modules, thereby enhancing the performance of entity extraction, medical coding, and summarisation components.

5.2.4 Tokenization and Label Alignment

Clinical text is tokenized using the BioBERT WordPiece tokenizer with a maximum sequence length of 512 tokens. For fine-tuning, BIO labels from the i2b2 annotation format are aligned to the subword token sequence using the following rule: the first subword token of each annotated word receives the B- or I- label; subsequent subword tokens of the same word receive the X label (indicating continuation) and are excluded from loss computation during training.

For documents exceeding 512 tokens, a sliding window approach with a window size of 512 tokens and a stride of 256 tokens is applied. Duplicate predictions in the overlap region (tokens 257–512 of the first window and tokens 1–256 of the second window) are resolved by retaining the prediction from the window where the token appears in the non-overlapping region, unless the overlap-region confidence is more than 15% higher, in which case the higher-confidence prediction is retained.

5.2.5 Data Augmentation

To address class imbalance in the Allergy and Dosage categories, two augmentation strategies are applied:

(i) **Entity Synonym Replacement:** Identified entities are replaced with clinical synonyms from SNOMED CT synonym lists, generating new training examples with the same annotations.

(ii) **Back-Translation:** Selected sentences containing underrepresented entities are translated to French and back to English using a neural machine translation model, producing paraphrased variants that preserve clinical meaning.

5.3 NER Model Architecture Design

The effectiveness of clinical information extraction largely depends on the architecture of the underlying Named Entity Recognition (NER) model and its ability to capture contextual relationships within complex medical text. Clinical narratives contain domain-specific terminology, abbreviations, and varying contextual dependencies that require models capable of understanding semantic meaning beyond simple pattern matching. Transformer-based language representations combined with sequence modelling techniques provide a robust approach for identifying clinically relevant entities while preserving contextual information. The proposed architecture integrates contextual embedding generation, sequence-level prediction mechanisms, and contextual assertion analysis to improve extraction accuracy and reliability.

5.3.1 BioBERT Encoder

The NER encoder is BioBERT-base (cased), comprising 12 transformer encoder layers, 12 attention heads per layer, a hidden dimension of 768, and a total of 110 million parameters. The model processes tokenized input and produces a sequence of contextual

embeddings of dimension 768 for each token. The `[CLS]` token embedding is not used for NER; only the per-token embeddings are passed to the classification head.

5.3.2 CRF Classification Head

A linear classification layer maps each 768-dimensional token embedding to a vector of logits over the label vocabulary (17 labels: B- and I- for each of 8 entity types, plus O). A Conditional Random Field (CRF) layer is applied on top of the linear layer to enforce label sequence constraints. The CRF layer learns a transition score matrix T where $T[i][j]$ represents the learned score of transitioning from label i to label j. During inference, the Viterbi algorithm decodes the optimal label sequence.

The CRF layer enforces the following hard constraints:

- I-TYPE labels cannot follow O labels without an intervening B-TYPE label.
- I-TYPE labels of one entity type cannot follow B-TYPE or I-TYPE labels of a different entity type.
- B-TYPE and I-TYPE labels of the same entity type can follow each other without restriction.

5.3.3 Negation Detection Module

The negation detection module is implemented as a rule-enhanced neural classifier. For each extracted entity, a context window of ± 5 tokens around the entity span is extracted from the tokenized document. A linear classifier operating on the BioBERT embeddings of the context window tokens predicts one of three assertion states: Affirmed, Negated, or Uncertain. The classifier is initialized with NegEx trigger phrase lists and fine-tuned on the i2b2 2010 assertion status annotations.

5.4 SNOMED CT Mapping Module Design

Standardized representation of extracted clinical concepts is essential for ensuring semantic interoperability and consistent interpretation of healthcare information across different systems. Mapping free-text medical entities to standardized terminology systems presents challenges because of variations in wording, contextual ambiguity, and the large number of available medical concepts. Efficient retrieval and ranking mechanisms are therefore required to identify the most appropriate standardized concepts

while maintaining accuracy and computational efficiency. The proposed SNOMED CT mapping framework employs semantic representation, candidate retrieval techniques, and confidence-based ranking mechanisms to generate reliable coding suggestions.

5.4.1 Concept Embedding Index Construction

The SNOMED CT concept description index is constructed as follows:

(i) All active concepts in the UK SNOMED CT release are extracted.

(ii) For each concept, its preferred term, fully specified name, and all active synonyms are concatenated into a description string.

(iii) FastText embeddings are computed for each description string using a character n-gram model ($n = 3$–6) pre-trained on a biomedical text corpus.

(iv) The resulting embedding vectors (dimension 300) are indexed in a FAISS `IndexFlatIP` (inner product) index for exact search, and a FAISS `IndexIVFPQ` index for approximate search.

5.4.2 Candidate Retrieval and Reranking

For each affirmed entity, the following two-stage retrieval process is performed:

- **Stage 1 — Candidate Retrieval:** The entity text is encoded using the FastText model to produce a query vector. The top-50 nearest neighbours (by inner product similarity) are retrieved from the FAISS index, producing 50 candidate SNOMED CT concepts.

- **Stage 2 — Reranking:** The 50 candidates are reranked using a cross-encoder model — a fine-tuned BioBERT model that takes as input the concatenation of the entity span (with its surrounding context) and the SNOMED CT concept description, and outputs a relevance score. The top-5 candidates by reranking score are returned as the final suggestions with associated confidence scores.

5.4.3 Confidence Score Calibration

Raw reranker scores are calibrated to produce reliable probability estimates using Platt scaling, fitting a logistic regression model on the reranker scores and binary relevance labels from a held-out validation set. Calibrated confidence scores are verified to be within 5% of empirical accuracy across five confidence deciles on the validation set.

5.5 Model Architecture and Experimental Setup

The ICDPS integrates multiple AI/ML model components, each addressing a distinct processing challenge. This section presents the architecture of each model component and the experimental setup adopted for training and evaluation.

5.5.1 Model Selection

Model selection was guided by three criteria: (i) clinical domain suitability, favouring models pre-trained on biomedical or clinical text; (ii) service integration feasibility, preferring managed AWS services; and (iii) published performance benchmarks on comparable tasks. Table 5.3 presents the candidate models considered and the selection rationale for each pipeline component.

Table 5.3: Model Selection Analysis for ICDPS Components

Component	**Candidates Considered**	**Selected Model and Rationale**
OCR Engine	AWS Textract; Tesseract 5; Google Cloud Vision	AWS Textract — superior table/form extraction; per-word confidence scoring; HIPAA-eligible
Layout Understanding	LayoutLMv3; LayoutXLM; DocFormer	LayoutLMv3 — state-of-the-art on PubLayNet/FUNSD; multi-modal text + layout + image
Medical NER	AWS Comprehend Medical; BioBERT; spaCy scispaCy	AWS Comprehend Medical — managed HIPAA-eligible; native InferSNOMEDCT API
Clinical LLM	AWS Bedrock Claude Sonnet; GPT-4; Llama 3	AWS Bedrock Claude Sonnet — best instruction following; lowest hallucination rate; HIPAA-eligible
Concept Embedding	SapBERT; BioBERT-NER; ClinicalBERT	SapBERT — purpose-designed for medical entity normalisation on SNOMED concepts
Vector Search	FAISS; Pinecone; Milvus	FAISS — open-source; GPU-accelerated; sub-100 ms retrieval

5.5.2 Model Architecture Design

LayoutLMv3 Architecture: LayoutLMv3 is a unified multi-modal pre-trained model for document understanding. The model architecture consists of a shared trans-

former encoder with 12 layers, 12 attention heads, and a hidden dimension of 768 (LayoutLMv3-base). Text tokens are represented as WordPiece embeddings summed with 1D position embeddings and 2D spatial layout embeddings derived from word bounding box coordinates. Image patches are encoded using a linear projection of 16×16 pixel patches from the document page image. Cross-modal attention between text token representations and image patch representations enables the model to associate text with its visual context on the page. In the ICDPS, LayoutLMv3 is fine-tuned for token classification into five layout-semantic categories: Title, Section Header, Body Text, Data Field, and Table Cell.

AWS Comprehend Medical NER: The `InferSNOMEDCT()` API operates on a clinical BERT-based sequence labeller pre-trained on MIMIC-III clinical notes, PubMed abstracts, and UMLS metathesaurus concept descriptions. The model applies a CRF output layer to produce entity span labels in BIO format. Each detected entity is associated with a SNOMED CT concept ID, a description string, a clinical category, and a detection confidence score.

AWS Bedrock Claude Sonnet: Claude Sonnet provides a 200,000-token context window, enabling processing of multi-page clinical documents without truncation. Three role-differentiated system prompts are constructed at runtime: a Clinician System Prompt for structured clinical summaries covering findings, diagnoses, and recommended actions; a Patient System Prompt for plain-English explanation avoiding medical jargon; and a Pharmacist System Prompt focusing on medication details, dosages, and contraindications.

5.5.3 Training Configuration

The LayoutLMv3 fine-tuning training configuration was established through a preliminary hyperparameter grid search. Table 5.4 presents the final training configuration adopted.

5.5.4 Experimental Workflow

The experimental workflow followed structured training, evaluation, and hyperparameter refinement cycles. In each cycle: (i) LayoutLMv3 was fine-tuned on the augmented training dataset; (ii) the fine-tuned model was evaluated on the validation set using token-level accuracy, precision, recall, and F1-score for each zone class; (iii) training

and validation loss curves were inspected for overfitting; (iv) hyperparameters were adjusted based on validation performance; and (v) the best checkpoint by macro F1-score was retained. For SNOMED mapping, layer-by-layer ablation experiments quantified the marginal contribution of each fallback layer to overall SNOMED code recall. UCS weighting coefficients were validated through threshold sensitivity analysis across the full confidence threshold range $[0.60, 0.80]$ for all 21 document type categories.

Table 5.4: LayoutLMv3 Fine-Tuning Training Configuration

Hyperparameter	**Value**
Base Model	`microsoft/layoutlmv3-base` (Hugging Face Hub)
Task	Token Classification — document layout segmentation
Training Epochs	15 (early stopping patience: 5 epochs)
Batch Size	8 (gradient accumulated over 4 steps; effective batch = 32)
Optimiser	AdamW
Initial Learning Rate	2×10^{-5}
Learning Rate Schedule	Cosine annealing with warm-up (warmup ratio = 0.1)
Weight Decay	0.01
Dropout Rate	0.1
Max Input Sequence Length	512 tokens
Image Resolution	224×224 pixels (ViT patch size 16×16)
Hardware	NVIDIA A10G GPU (AWS g5.xlarge)
Training Duration	Approximately 4.5 hours for 15 epochs
Framework	PyTorch 2.1 + Hugging Face Transformers 4.37

5.5.5 Validation Strategy

The primary validation strategy for SNOMED CT mapping accuracy employs leave-one-out cross-validation (LOOCV) across the 25 annotated documents, reporting precision, recall, and F1-score at the entity-code pair level. LOOCV was selected because the small dataset size makes k-fold partitions unstable for minority document categories. For LayoutLMv3 zone classification, stratified 5-fold cross-validation on the synthetic training dataset reports token-level macro F1-score. OCR accuracy is evaluated using character error rate (CER) and word error rate (WER) computed against manually verified ground-truth transcriptions of 15 representative document pages.

5.6 Inference and Post-Processing Design

The effectiveness of an intelligent clinical document processing system depends not only on model performance but also on the mechanisms used to optimize prediction quality and transform raw outputs into meaningful structured information. Efficient inference workflows require techniques that improve computational performance while maintaining accuracy and model stability during processing. Additional post-processing operations are necessary to refine extracted entities, remove inconsistencies, and ensure that generated outputs satisfy clinical and semantic requirements. The components presented in this section describe the optimization strategies, inference mechanisms, output refinement procedures, and structured data representation techniques adopted within the proposed Intelligent Clinical Document Processing System (ICDPS).

5.6.1 Optimisation Techniques

Gradient-based optimisation for LayoutLMv3 fine-tuning uses AdamW, which decouples weight decay from the adaptive gradient update mechanism, providing superior regularisation for transformer models. Weight decay of 0.01 is applied to all parameters except bias terms and layer normalisation parameters. Learning rate warm-up over the first 10% of training steps prevents early instability. Cosine annealing reduces the learning rate smoothly from its peak to near-zero, enabling fine-grained convergence. Gradient clipping with maximum norm 1.0 prevents exploding gradients. Mixed-precision training (FP16) reduces GPU memory consumption by approximately 40% and enables a larger effective batch size. Early stopping with patience of 5 validation epochs prevents overfitting.

5.6.2 Loss Functions and Evaluation Logic

The LayoutLMv3 token classification objective uses weighted cross-entropy loss with class weights inversely proportional to class frequency, addressing the imbalance between dominant Body Text tokens and sparse Table Cell or Section Header tokens. The weighted loss $\mathcal{L}_w$ is:

$$\mathcal{L}_w = -\sum_{c} [w_c \cdot y_c \cdot \log(p_c)] \quad \text{where} \quad w_c = \frac{N_{\text{total}}}{N_{\text{classes}} \cdot N_c} \tag{5.1}$$

The Unified Confidence Score (UCS) is computed using document-type-dependent weighted linear combinations. For standard NHS documents (discharge summaries, specialist letters):

$$\text{UCS} = 0.40 \times \text{textract\_conf} + 0.20 \times \text{snomed\_conf} + 0.40 \times \text{llm\_conf} \tag{5.2}$$

For ambulance and NHS 111 reports, where all-caps tables render SNOMED mapping unreliable:

$$\text{UCS} = 0.50 \times \text{textract\_conf} + 0.50 \times \text{llm\_conf} \tag{5.3}$$

The UCS is compared against document-type-specific thresholds defined in `config/document_type_config.py`, which defines 21 NHS document categories with per-type review thresholds ranging from 0.62 to 0.72.

5.6.3 Inference Pipeline

The production inference pipeline is implemented as a Python class (`IcdpsInferencePipeline`) that encapsulates the complete processing workflow. The pipeline accepts a document path as input and executes the following steps in sequence: document format detection, text extraction (direct or OCR), text normalization, sliding-window tokenization, NER inference, entity span decoding, negation detection, role categorization, summarization, SNOMED CT candidate retrieval, and reranking. The pipeline returns a structured JSON object containing the document metadata, extracted entities, summary, and SNOMED CT suggestions.

5.6.4 Post-Processing Rules

The following post-processing rules are applied to the raw NER output:

- **Contiguous Entity Merging:** Adjacent entities of the same type separated only

by whitespace or the conjunction “and” are merged into a single entity span (e.g., “left” + “ventricular” + “hypertrophy” → “left ventricular hypertrophy”).

- **Minimum Entity Length Filter:** Entity spans of fewer than 2 non-whitespace characters are discarded as likely tokenization artefacts.
- **Duplicate Entity Deduplication:** Identical entity text appearing more than once in the document is deduplicated; only the first occurrence with its associated SNOMED CT suggestion is retained in the structured output. All occurrences are annotated in the document view.
- **Negated Entity Exclusion:** Entities with negation status Negated are excluded from SNOMED CT code suggestion. They are retained in the display but are clearly marked and do not appear in the exported coding report.

5.6.5 Output Schema

The inference pipeline produces a structured JSON output conforming to the following schema:

```
{
  "document_id": string,
  "document_type": string,
  "extraction_timestamp": ISO8601,
  "entities": [{
    "entity_id": string,
    "text": string,
    "category": string,
    "assertion": string,
    "role_action": string,
    "start_char": int,
    "end_char": int,
    "snomed_suggestions": [{
      "concept_id": string,
      "preferred_term": string,
      "confidence": float
```

```
    }]
  }],
  "summary": string,
  "processing_metadata": {
    "model_version": string,
    "processing_time_ms": int
  }
}
```

Summary

This chapter presented the detailed technical design of the ICDPS, covering dataset organization and characteristics, the text preprocessing and augmentation pipeline, the BioBERT-CRF NER model architecture, the two-stage SNOMED CT candidate retrieval and reranking system, and the inference pipeline with post-processing rules. The detailed design established the precise technical specifications required to implement the system, providing a clear bridge between the high-level architectural blueprint in Chapter 4 and the implementation described in Chapter 6. Key design decisions — including sliding-window inference for long documents, CRF-enforced label constraints, and confidence score calibration — are justified in terms of their contribution to system accuracy and reliability.

Chapter 6

Implementation Details

CHAPTER 6

IMPLEMENTATION DETAILS

The successful realization of an intelligent clinical document processing system requires the conversion of architectural designs and theoretical models into a functional implementation framework. Multiple components including document ingestion, preprocessing, information extraction, semantic understanding, and standardized coding must operate together to achieve reliable and efficient system behaviour. The Intelligent Clinical Document Processing System (ICDPS) integrates various technologies, machine learning models, and software tools to implement the proposed processing pipeline. The topics presented in this chapter describe the development environment, implementation workflow, model training procedures, software structure, evaluation mechanisms, and challenges encountered during system development

6.1 Implementation Workflow

The implementation of the ICDPS followed a phased workflow aligned with the iterative SDLC adopted for the project. Each phase produced verifiable outputs that were validated before proceeding to the next phase. The workflow comprised ten sequential steps from environment setup through post-deployment monitoring.

6.1.1 Environment Setup

The development environment was configured on a workstation running Ubuntu 22.04 LTS equipped with an NVIDIA RTX 3090 GPU (24 GB VRAM). The following installation sequence was followed:

1. Python 3.10 was installed via the deadsnakes PPA and a virtual environment was created using `python3.10 -m venv icdps_env` to isolate project dependencies.

2. CUDA 11.8 and cuDNN 8.6 were installed from the NVIDIA developer portal to enable GPU-accelerated PyTorch operations.

3. PyTorch 2.0 with CUDA 11.8 support was installed.

4. The Hugging Face Transformers library (v4.38), Datasets library (v2.17), and PEFT library (v0.8) were installed for model fine-tuning and evaluation.

5. Tesseract 5.0 OCR engine was installed.

6. PyMuPDF (v1.23), FAISS-GPU (v1.7.4), Flask (v3.0), SQLAlchemy (v2.0), and PostgreSQL 16 client libraries were installed.

7. Docker 24 and Docker Compose v2 were installed for containerized deployment preparation.

8. Jupyter Lab was configured for exploratory data analysis and training experiment notebooks; Visual Studio Code with the Python and Pylance extensions served as the primary IDE for production code development.

All dependencies were recorded in a `requirements.txt` file with pinned version numbers to ensure environment reproducibility. A Docker image encapsulating the complete runtime environment was built for deployment.

6.1.2 Data Collection

The quality and diversity of the input dataset play a significant role in determining the overall performance of an intelligent clinical document processing system. Since healthcare documentation exhibits substantial variation in formatting, image quality, document structure, and terminology usage, it was necessary to construct a dataset that reflected realistic NHS clinical scenarios. The data collection process therefore aimed to capture representative examples of documents encountered within routine clinical workflows while ensuring compliance with ethical and privacy considerations.

The dataset used in this research consisted of 40 NHS clinical documents obtained from Easthampstead Surgery under a formal academic research data-sharing agreement. All data acquisition procedures were conducted in accordance with institutional research guidelines to ensure secure handling of clinical information. The document set included both digitally generated clinical records and scanned paper documents, thereby reflecting the heterogeneous nature of healthcare documentation systems commonly found within NHS environments.

Physical paper documents were digitised using a Canon imageRUNNER scanner at a resolution of 300 dpi. This resolution was selected because it provides an appropriate balance between image quality and computational efficiency, while also aligning with commonly recommended standards for OCR applications. Digital documents already stored within the clinical practice's document management system were exported directly

as PDF files, thereby preserving their native formatting and avoiding unnecessary quality degradation introduced through repeated scanning processes.

Although the collected real-world dataset was suitable for evaluation, the limited number of available annotated documents created challenges for training data-intensive document understanding models such as LayoutLMv3. To address this limitation, a synthetic document generation pipeline was implemented using the Python ReportLab library. The objective of this process was to generate structurally realistic NHS clinical documents while introducing controlled variability in document appearance.

The synthetic generation pipeline produced approximately 800 NHS clinical letter images based on representative templates and layouts observed in the real document collection. To simulate realistic scanning and document quality conditions, generated documents were further processed using ImageMagick-based degradation transformations. These transformations introduced variations including image blur, brightness shifts, contrast changes, and artificial noise. The resulting synthetic documents enabled the model to learn robustness against variations commonly observed in practical healthcare environments.

Ground-truth clinical coding labels were required to evaluate extraction accuracy and support supervised learning tasks. Consequently, SNOMED CT annotations were generated for the 25 evaluation documents through manual review by a clinical domain expert using the NHS SNOMED CT browser. Manual annotation ensured that extracted concepts were clinically accurate and reflected appropriate code assignments according to NHS terminology standards.

6.1.3 Data Preprocessing

Raw clinical documents frequently contain artefacts and inconsistencies that negatively affect OCR accuracy and subsequent natural language processing stages. Variations in scanning orientation, uneven illumination, background noise, and photocopy degradation can substantially reduce text recognition quality. Therefore, a dedicated preprocessing pipeline was implemented to improve document readability and standardise image characteristics before OCR processing.

The image preprocessing pipeline was implemented as a standalone Python module (`preprocessing/image_pipeline.py`) containing a sequence of OpenCV-based enhancement operations. The preprocessing architecture was designed to minimise image quality

issues while preserving the structural integrity of textual content.

The first stage involved adaptive thresholding to improve text contrast and address illumination inconsistencies. The implementation used OpenCV's `cv2.adaptiveThreshold()` function configured with:

- Method: `cv2.ADAPTIVE_THRESH_GAUSSIAN_C`
- Block size: 11
- Constant parameter: 2

Unlike global thresholding approaches that apply a single threshold across an entire image, adaptive thresholding computes local threshold values based on neighbouring pixel intensity distributions. Specifically, Gaussian-weighted averaging across surrounding 11×11 pixel regions was used to determine threshold values dynamically for each image location.

This approach proved particularly effective for handling:

- Uneven lighting conditions
- Curved document pages
- Shadow effects from handheld photography
- Localised brightness variations

Following thresholding, morphological filtering was applied to reduce image noise while preserving text structure. Morphological processing used OpenCV's `cv2.morphologyEx(MORPH_CLOSE)` with a 2×2 rectangular structuring element. The closing operation performs dilation followed by erosion, enabling the removal of isolated dark regions and the filling of small discontinuities within character strokes. This operation improved OCR readability by:

- Removing background speckles
- Reducing photocopy artefacts
- Connecting fragmented text strokes
- Preserving text continuity

The final image preprocessing stage involved document deskewing. Documents captured from scanners or mobile devices frequently contain rotational distortions that can reduce OCR effectiveness. Text orientation was estimated using contour analysis:

1. Contours were detected using `cv2.findContours()`
2. Bounding boxes were computed using `cv2.minAreaRect()`
3. Orientation angles were extracted from detected text regions
4. Median angle estimation was used to obtain a stable page rotation estimate

After angle estimation, images were rotated using `cv2.warpAffine()` with bicubic interpolation to minimise quality loss during geometric transformation. To avoid incorrect corrections caused by sparse text regions or near-empty pages, rotation angles were constrained to ±15. This safeguard prevented extreme corrections that could otherwise distort document content.

6.1.4 Feature Engineering

Feature engineering was performed to transform OCR outputs into structured and semantically meaningful representations suitable for downstream extraction and coding tasks. Since OCR systems frequently introduce inconsistencies such as broken sentence structures, irregular spacing, and encoding artefacts, additional normalisation procedures were necessary before passing extracted text into subsequent NLP components.

Feature engineering operations were implemented within the `extraction/text_normaliser.py` module and executed sequentially.

The first transformation involved Unicode normalisation using `unicodedata.normalize('NFKD')`. Unicode decomposition reduced inconsistencies arising from special symbols and character encoding differences across document sources.

The second stage expanded NHS-specific abbreviations using a manually curated dictionary containing 847 clinical abbreviations. Clinical documentation frequently contains shorthand terminology such as:

- BP → Blood Pressure
- SOB → Shortness of Breath
- HR → Heart Rate

Expansion reduced ambiguity and improved downstream entity extraction performance. Subsequent transformations included:

- **Page boundary insertion:** Explicit markers were inserted between concatenated multi-page text blocks to preserve structural information and prevent contextual mixing across pages.
- **Whitespace normalisation:** Repeated spaces, tabs, and newline characters were collapsed into single spaces to ensure consistent text formatting.
- **Sentence boundary correction:** OCR systems often incorrectly split sentences across multiple lines. Sentence boundary normalisation inserted punctuation at locations where line breaks interrupted incomplete sentence structures.

For semantic clinical concept matching, extracted entities identified by AWS Comprehend Medical were passed into the SapBERT pipeline. Clinical terms were tokenised and encoded into 768-dimensional biomedical embeddings representing semantic relationships between concepts. These embeddings were stored and queried using the FAISS nearest-neighbour search framework, enabling efficient retrieval of semantically related clinical concepts and improving SNOMED code matching accuracy.

6.1.5 Model Selection

Selection of suitable models for each stage of the pipeline was based on comparative benchmarking and performance evaluation. Multiple candidate approaches were assessed to identify models capable of achieving high accuracy while remaining computationally practical for deployment.

OCR evaluation demonstrated that AWS Textract substantially outperformed Tesseract 5.0 across both standard and degraded NHS documents.

- Standard document images: AWS Textract 94.7% vs. Tesseract 82.4% (+12.3 pp)
- Degraded scanned documents: AWS Textract 87.2% vs. Tesseract 67.4% (+19.8 pp)

The stronger performance of Textract can largely be attributed to its ability to capture document layout information and contextual relationships beyond simple character recognition.

For document understanding tasks, LayoutLMv3 was evaluated against LayoutLMv2 and a rule-based baseline approach. Results demonstrated:

- LayoutLMv3: Macro F1 = 0.891
- LayoutLMv2: Macro F1 = 0.824
- Rule-based classifier: Macro F1 = 0.712

LayoutLMv3 achieved the highest performance because of its unified multimodal architecture combining textual and visual representations.

For clinical entity extraction and SNOMED mapping, AWS Comprehend Medical InferSNOMEDCT() was compared against a BioBERT-based NER pipeline. Performance results showed:

- Precision: Comprehend Medical = 0.823; BioBERT = 0.761
- Recall: Comprehend Medical = 0.714; BioBERT = 0.742

Although BioBERT achieved slightly higher recall, higher precision was prioritised because false-positive clinical codes can introduce clinically disruptive outcomes.

6.1.6 Model Training

The document understanding component required domain adaptation to accurately identify structural patterns present within NHS clinical documentation. Consequently, a dedicated fine-tuning stage was performed using the combined real and synthetic dataset.

The LayoutLMv3 fine-tuning process was implemented using the Hugging Face Trainer API through the script `training/finetune_layoutlmv3.py`. A custom `DocumentTokenClassificationDataset` class was developed to load document images, OCR text, bounding box coordinates, and annotation labels stored in JSON format.

The final training dataset consisted of approximately 850 document images derived from both real and synthetic sources. Dataset allocation included:

- Training set: 680 documents (80%)
- Validation and testing: remaining 170 documents (20%)

Training was executed using an AWS g5.xlarge instance equipped with GPU acceleration. Training proceeded for a maximum of 15 epochs over approximately 4.5 hours. Training convergence behaviour demonstrated stable learning characteristics:

- Epoch 1 loss: 1.423 → Epoch 12 loss: 0.187
- Minimum validation loss: 0.214 at epoch 10

Following epoch 12, validation loss began increasing while training loss continued decreasing, indicating the onset of overfitting. Early stopping with a patience value of five epochs was therefore employed, terminating training at epoch 15 and preserving the model parameters corresponding to the lowest validation loss. This strategy ensured improved generalisation performance while reducing unnecessary computational cost.

6.1.7 Model Evaluation

Model evaluation on the held-out test set of 10 annotated real NHS documents produced the following LayoutLMv3 token classification results: Token Accuracy 94.3%, Macro F1-Score 0.887, Precision 0.901, Recall 0.874. Per-class F1-scores are presented in Table 6.1.

Table 6.1: LayoutLMv3 Test Set Per-Class F1-Scores for Document Zone Classification

Document Zone Class	**Precision**	**Recall**	**F1-Score**
Title	0.978	0.961	0.969
Section Header	0.934	0.912	0.923
Body Text	0.941	0.958	0.949
Data Field	0.889	0.843	0.865
Table Cell	0.862	0.797	0.828
Macro Average	**0.921**	**0.894**	**0.907**

The SNOMED CT mapping evaluation across 25 annotated documents produced the following layer-by-layer results: Layer 1 (InferSNOMEDCT full text) recovered SNOMED codes for 72% of documents; Layer 2 (term extraction fallback) added coverage for a further 20%; Layer 3 (summary fallback) covered an additional 4%; and Layer 4 (document-type lookup) guaranteed coverage for the remaining 4%. Overall cascade SNOMED entity-level Precision: 0.847, Recall: 0.793, F1-Score: 0.819. Table 6.2 presents the SNOMED mapping performance by layer.

Table 6.2: SNOMED CT Mapping Performance Evaluation by Layer

Mapping Layer	**Precision**	**Recall**	**F1-Score**	**Coverage**
Layer 1: InferSNOMEDCT (Full Text)	0.871	0.782	0.824	72%
Layer 2: Term Extraction Fallback	0.824	0.693	0.752	92%
Layer 3: Summary Fallback	0.796	0.654	0.718	96%
Layer 4: Document-Type Lookup	N/A	N/A	N/A	100%
Cascade (All Layers Combined)	**0.847**	**0.793**	**0.819**	**100%**

6.1.8 Model Optimization

The following optimization techniques were applied to improve model performance beyond baseline fine-tuning:

- **Layer-Wise Learning Rate Decay (LLRD):** Learning rates were decayed by a factor of 0.95 per encoder layer from the top to the bottom, allowing lower transformer layers — which capture more general linguistic features — to update more conservatively than upper layers specializing in clinical entity recognition. This improved validation F1 by 0.5 percentage points.

- **Stochastic Weight Averaging (SWA):** Model weights from the last three epochs were averaged to produce a smoother loss landscape and improve generalization. SWA improved test F1 by 0.3 percentage points compared to the best single-epoch checkpoint.

- **Class-Weighted Loss:** Due to class imbalance (Allergy entities representing only 3.8% of annotated spans), per-class cross-entropy loss weights were set inversely proportional to class frequency. This improved Allergy entity F1 from 0.71 to 0.79.

- **Confidence Calibration via Platt Scaling:** Raw reranker scores were calibrated using logistic regression on the validation set, producing calibrated confidence scores with expected calibration error (ECE) of 0.032 on the test set.

6.1.9 Model Deployment

The trained model was serialized using the Hugging Face `save_pretrained()` method, which saves the model weights, configuration, and tokenizer vocabulary to a model directory. The production deployment comprises:

- **NLP Worker Container:** A Docker container based on the `pytorch/pytorch:2.0.1-cuda11.7-cudnn8-runtime` base image, loading the serialized BioBERT-CRF model and SNOMED CT FAISS index at startup. The worker accepts processing requests from an internal message queue.
- **Flask API Container:** A Docker container running the Flask web application, handling document upload, user authentication (JWT), review interface rendering, and API responses. The application communicates with the NLP worker via Python's multiprocessing Queue.
- **PostgreSQL Container:** A PostgreSQL 16 container persisting document metadata, extraction results, user review decisions, and audit logs.
- **Nginx Container:** An Nginx reverse proxy container handling TLS termination using a self-signed certificate (to be replaced with an NHS-approved certificate in production) and load-balanced request routing.

The complete deployment stack is defined in a `docker-compose.yml` file and can be started with a single command: `docker-compose up -d`. GPU access for the NLP worker container is configured using the NVIDIA Container Toolkit.

6.1.10 Monitoring and Maintenance

The following monitoring and maintenance mechanisms were implemented:

- **Application Logging:** All processing events — document receipt, OCR completion, NER inference, SNOMED retrieval, and user code decisions — are logged using Python's `logging` module at INFO and DEBUG levels. Log entries include timestamps, document IDs, processing durations, and entity counts. Logs are persisted to the PostgreSQL database for structured querying.
- **Performance Monitoring:** Inference latency per document is tracked and stored. A monitoring dashboard built with Flask-Admin displays average latency, daily

document throughput, and per-category entity extraction counts over configurable time windows.

- **Model Degradation Detection:** A scheduled weekly evaluation job reruns the model on a fixed reference document set of 20 documents and alerts the administrator if average F1 drops below 0.82, indicating potential model drift.
- **Model Update Workflow:** New training data can be added to the fine-tuning corpus and the model retrained using the training script with a single command. Model versioning is tracked using Git tags, and the deployment container image is rebuilt and pushed to the internal container registry automatically via a GitHub Actions CI/CD pipeline.

6.2 Code Structure and Project Organization

The implementation of a multi-component intelligent clinical document processing system required a structured software architecture to ensure maintainability, scalability, and modular development. Since the system integrates OCR, document understanding, clinical entity extraction, semantic retrieval, and summarisation modules, organising the project into clearly defined components was essential to reduce code complexity and support independent development and testing. The project structure was therefore designed using a modular approach, where related functionalities were grouped into dedicated directories and reusable components. This organisation facilitated easier debugging, simplified integration of new features, and improved overall software maintainability throughout the development lifecycle.

6.2.1 Project Directory Structure

The organisation of source code and project assets is a critical aspect of software engineering, particularly for systems consisting of multiple interdependent modules and machine learning components. As the proposed framework integrates image preprocessing, OCR, document understanding, entity extraction, semantic retrieval, and cloud-based services, maintaining a clear directory structure was essential to ensure efficient development and ease of maintenance. A well-defined project hierarchy improves code readability, simplifies debugging processes, supports modular implementation, and enables future extensions without affecting existing functionality. The project directory structure was

therefore designed to separate functional components into dedicated modules, allowing independent development and facilitating clearer interaction between different stages of the processing pipeline. The overall directory organisation and the purpose of each major component are presented below.

```
NLP-uk/
|-- app.py                          # Flask application entry point
|-- document_handler.py             # Multi-format document ingestion
|-- preprocessing/
|   |-- image_pipeline.py           # OpenCV 3-stage preprocessing
|   |-- text_normaliser.py          # Text cleaning and normalisation
|-- ocr/
|   |-- textract_client.py          # AWS Textract API wrapper
|   |-- confidence_router.py        # Tier 1 / Tier 2 routing logic
|-- layout/
|   |-- layoutlmv3_refiner.py       # LayoutLMv3 inference service
|   |-- zone_classifier.py          # Document zone classification
|-- extraction/
|   |-- snomed_mapper.py            # 4-layer SNOMED CT mapping cascade
|   |-- phi_detector.py             # Comprehend Medical PHI detection
|   |-- regex_extractor.py          # ICD codes, meds, demographics
|   |-- entity_extractor.py         # detect_entities_v2 wrapper
|-- summarisation/
|   |-- bedrock_client.py           # AWS Bedrock Claude API wrapper
|   |-- prompt_builder.py           # Role-specific prompt construction
|-- confidence/
|   |-- ucs_calculator.py           # Unified Confidence Score logic
|-- config/
|   |-- document_type_config.py     # 21 NHS document type thresholds
|-- training/
|   |-- finetune_layoutlmv3.py      # LayoutLMv3 training script
|   |-- dataset.py                  # DocumentTokenClassificationDataset
|   |-- evaluate.py                 # Evaluation metrics computation
|-- frontend/                       # React NHS Portal UI
|   |-- src/
```

```
|   |   |-- components/            # React UI components
|   |   |-- App.jsx                # Main application component
|   |-- package.json
|-- docker/
|   |-- Dockerfile                 # Multi-stage Docker build
|   |-- docker-compose.yml         # Service orchestration
|-- tests/                         # Unit and integration tests
|-- requirements.txt               # Python dependencies (pinned)
|-- README.md                      # Project documentation
```

6.2.2 Source Code Organization

Each `src/` subdirectory contains a dedicated module with a clear single responsibility. The ingestion module exposes a `DocumentIngester` class with a unified `ingest(path)` $\rightarrow$ `str` interface regardless of input format (PDF or image). The extraction module exposes an `NerEngine` class with a `predict(text)` $\rightarrow$ `List[Entity]` interface. All inter-module communication uses typed Python dataclasses (`Entity`, `SnomedSuggestion`, `ExtractionResult`) defined in `src/utils/schemas.py`, ensuring consistent data contracts across the pipeline.

Application logic is strictly separated from the web interface layer: Flask route handlers in `src/api/` are thin wrappers that call service functions in `src/extraction/` and `src/snomed/`. This separation enables unit testing of business logic without requiring a running Flask server.

6.2.3 Model Storage and Versioning

Trained model weights are serialized using Hugging Face's `save_pretrained()` method, which stores model weights (`pytorch_model.bin`), configuration (`config.json`), and tokenizer vocabulary (`vocab.txt`) in a self-contained directory. Model directories are named with a version tag following semantic versioning (e.g., `ner_biobert_v1.2.0`). The active model version is specified in `config.yaml` and loaded at application startup.

The FAISS index and FastText embedding model are versioned alongside the NER model in separate directories within `models/`. Index files are stored as binary `.faiss` files and loaded into memory at startup. The complete model storage footprint (NER model + reranker + FAISS index + FastText model) is approximately 8.2 GB.

6.2.4 Configuration and Dependency Management

All configurable parameters — model paths, database connection strings, processing thresholds, SNOMED CT index paths, logging levels, and API port numbers — are defined in a single `config.yaml` file loaded at application startup using the PyYAML library. Environment-specific overrides (e.g., production database credentials) are provided through environment variables injected by Docker Compose, avoiding hardcoded secrets in the codebase.

Python dependencies are managed via `requirements.txt` with pinned version numbers. A `Makefile` provides convenience commands for common development tasks: `make install` (create virtual environment and install dependencies), `make train` (run training scripts), `make test` (run test suite), `make deploy` (build and start Docker containers), and `make evaluate` (run evaluation on reference document set).

6.3 Libraries, Frameworks, and Tools Used

The implementation of the proposed system required the integration of multiple libraries, frameworks, and development tools spanning document processing, machine learning, cloud services, and deployment infrastructure. Since no single technology could support all functional requirements of the system, a collection of specialised tools was selected based on factors including performance, compatibility, scalability, and ease of integration. The following sections present the major technologies employed during implementation and discuss their specific roles within the overall architecture.

6.3.1 Programming Language

Python 3.10 was selected as the implementation language. Python's extensive ecosystem of NLP, ML, and scientific computing libraries (transformers, PyTorch, scikit-learn, NumPy, pandas) provides a complete toolkit for all pipeline components. Python's dynamic typing and interactive notebook environment (Jupyter Lab) accelerated exploratory development. PEP 8 style guidelines and type hints were enforced throughout the codebase using `flake8` and `mypy`.

6.3.2 AIML Libraries and Frameworks

Artificial Intelligence and Machine Learning libraries formed the core computational components of the proposed system, enabling document understanding, entity extraction, semantic search, and large language model integration. Multiple libraries were incorpo-

rated to support different stages of the processing pipeline, ranging from OCR preprocessing and biomedical embedding generation to model training and inference tasks. Selection of these frameworks was guided by model performance, availability of domain-specific pretrained models, and interoperability with cloud-based healthcare services. Table 6.3 summarises the AIML libraries and frameworks used in the implementation together with their primary functions within the system.

6.3.3 Development Tools

A range of software development tools was utilised throughout the implementation process to support coding, debugging, version control, dependency management, and project coordination activities. These tools contributed to improving development efficiency and maintaining consistency across different stages of implementation. Furthermore, they facilitated collaborative development practices and streamlined testing and integration procedures. Table 6.4 presents the development tools employed and outlines their specific roles within the project workflow.

6.3.4 Deployment and Environment Management Tools

Deployment and environment management tools were required to ensure reproducible execution environments and support scalable deployment of the proposed system. Since machine learning applications frequently involve multiple dependencies, library versions, and cloud-based resources, maintaining consistency across development and deployment stages was particularly important. Appropriate tools were therefore selected to manage virtual environments, containerisation, cloud infrastructure, and execution workflows. Table 6.5 summarises the deployment and environment management tools used and their contributions to the implementation process.

6.4 Implementation Practices and Coding Standards

The development of a large-scale AI-driven healthcare application requires the adoption of consistent implementation practices and coding standards to maintain software quality and long-term maintainability. As the system consists of multiple interconnected modules involving document processing, machine learning models, and cloud-based services, inconsistent coding approaches could introduce integration difficulties and increase maintenance overhead. Consequently, a set of development practices and coding standards was followed throughout implementation to ensure readability, modularity, error

Table 6.3: AIML Libraries and Frameworks Used

Library / Framework	**Version**	**Purpose in ICDPS**
PyTorch	2.0.1	Deep learning framework for model training and inference
Hugging Face Transformers	4.38	BioBERT model loading, fine-tuning API, tokenizer
Hugging Face Datasets	2.17	Efficient dataset loading, batching, and caching
TorchCRF	1.1.0	CRF layer implementation for label sequence decoding
seqeval	1.2.2	NER evaluation metrics (entity-level F1, precision, recall)
FAISS-GPU	1.7.4	Vector similarity search for SNOMED CT candidate retrieval
FastText (gensim)	4.3.0	SNOMED CT concept description embeddings
PyMuPDF	1.23	Direct text extraction from PDF documents
pytesseract	0.3.10	Python binding for Tesseract 5.0 OCR engine
Pillow	10.2	Image preprocessing for OCR (binarization, deskewing)
scikit-learn	1.4	Platt scaling calibration, data splitting, baseline models
spaCy	3.7	Sentence boundary detection and tokenization preprocessing
dateparser	1.2	Date expression normalization
NumPy	1.26	Numerical array operations
pandas	2.2	Tabular data processing and analysis

Table 6.4: Development Tools Used

Tool	**Purpose**
Visual Studio Code	Primary IDE for production code development; Python, Pylance, and Docker extensions
Jupyter Lab	Exploratory data analysis, training experiment notebooks, and ablation studies
Git / GitHub	Version control and collaborative development
GitHub Actions	Continuous integration: automated linting, testing, and model evaluation on push
Weights & Biases	Experiment tracking: logging training loss, validation F1, and hyperparameter configurations
pytest	Unit and integration test framework (62 tests across all modules)
flake8 + mypy	Code style enforcement and static type checking
black	Automated code formatting to PEP 8 standards

Table 6.5: Deployment and Environment Management Tools

Tool	**Version**	**Purpose**
Docker	24.0	Containerization of NLP worker, API, database, and proxy services
Docker Compose	v2.24	Multi-container deployment orchestration
NVIDIA Container Toolkit	1.14	GPU passthrough for NLP worker container
Nginx	1.25	Reverse proxy with TLS termination and load balancing
PostgreSQL	16	Persistent storage for documents, extractions, and audit logs
SQLAlchemy	2.0	ORM for database interaction
Flask	3.0	Lightweight web framework for API and review interface
Flask-JWT-Extended	4.6	JWT-based API authentication

handling consistency, and efficient software design.

6.4.1 Development Best Practices

The following development best practices were consistently applied throughout the project:

- **Modular Programming:** Each pipeline module was implemented as an independent Python class with a clearly defined public interface. Inter-module coupling was minimized by communicating exclusively through shared dataclass schemas.
- **Incremental Testing:** Unit tests were written alongside implementation for each module, following a test-driven development (TDD) approach for critical functions such as BIO label alignment and SNOMED CT score calibration. A continuous integration pipeline ran the full test suite on every code push.
- **Version Control:** All code changes were committed to the GitHub repository with descriptive commit messages following the Conventional Commits specification. Feature branches were used for new capabilities; pull requests were reviewed before merging to the main branch.
- **Error Handling:** All I/O operations (file reading, OCR, database access, model inference) were wrapped in try-except blocks with specific exception types. Processing failures at any pipeline stage were logged with full stack traces and routed to the error queue without propagating exceptions to the user interface.
- **Logging:** The Python `logging` module was configured at module level with consistent log formatting: `[TIMESTAMP] [MODULE] [LEVEL] MESSAGE`. DEBUG-level logs recorded intermediate processing states (entity spans, tokenization outputs) to facilitate troubleshooting without impacting production performance.

6.4.2 Naming Conventions

The following naming conventions were applied throughout the project:

- **Files:** `snake_case` for Python files (`ner_engine.py`, `snomed_mapper.py`); `kebab-case` for configuration and Docker files (`docker-compose.yml`, `config.yaml`).
- **Variables and Functions:** `snake_case` following PEP 8 (`entity_spans`, `predict_entities`, `build_snomed_index`).

- **Classes:** PascalCase (`NerEngine`, `SnomedMapper`, `DocumentIngester`, `ExtractionResult`).
- **Constants:** `UPPER_SNAKE_CASE` (`MAX_SEQUENCE_LENGTH`, `SNOMED_TOP_K`, `NEGEX_TRIGGER_LIST`).
- **Model Directories:** Versioned names with component prefix (`ner_biobert_v1.2.0`, `reranker_biobert_v1.0.0`).
- **Database Tables:** Plural `snake_case` (`documents`, `entities`, `snomed_suggestions`, `audit_logs`).

6.4.3 Modular Development and Maintainability

The ICDPS codebase achieves maintainability through three design principles. First, each module in `src/` has a single well-defined responsibility and exposes a minimal public API, reducing the surface area of interface changes. Second, all configuration values are externalized to `config.yaml` — no magic numbers or hardcoded paths appear in the source code — enabling configuration changes without code modification. Third, the pipeline is implemented using the Strategy design pattern for the document ingestion module: PDF extraction and OCR extraction are interchangeable strategies implementing a common `DocumentExtractor` interface, enabling straightforward addition of new document format strategies (e.g., DICOM text extraction) without modifying the pipeline orchestration code.

6.5 Model Evaluation Metrics

The ICDPS uses the following evaluation metrics, selected for their appropriateness to sequence labelling and information retrieval tasks. Table 6.6 presents the evaluation metrics used in the ICDPS.

6.6 Difficulties Encountered and Strategies Used to Tackle Them

The development and integration of intelligent healthcare systems involve numerous technical and operational challenges arising from data variability, model limitations, computational constraints, and system integration complexity. During the implementation of the proposed framework, several issues were encountered across different stages of the

Table 6.6: Evaluation Metrics Used in ICDPS

Metric	**Formula**	**Application**	**Justification**
Precision (P)	$\frac{\text{TP}}{\text{TP} + \text{FP}}$	NER entity extraction	Measures the fraction of predicted entities that are correct; critical for avoiding false clinical coding.
Recall (R)	$\frac{\text{TP}}{\text{TP} + \text{FN}}$	NER entity extraction	Measures the fraction of true entities captured; critical for ensuring completeness of clinical records.
F1-Score	$\frac{2PR}{P + R}$	NER primary metric	Harmonic mean balancing precision and recall; standard metric for NER tasks.
Precision@1	$\frac{\text{Correct in top-1}}{\text{Total}}$	SNOMED CT mapping	Proportion of cases where the first-ranked code is correct; measures direct utility for coders.
Precision@3	$\frac{\text{Correct in top-3}}{\text{Total}}$	SNOMED CT mapping	Proportion where correct code appears in top-3; reflects practical suggestion utility.
Precision@5	$\frac{\text{Correct in top-5}}{\text{Total}}$	SNOMED CT mapping	Proportion where correct code appears in top-5; upper-bound utility measure.
ECE	$\sum \frac{\lvert acc(b) - conf(b) \rvert}{B}$	Confidence calibration	Expected Calibration Error; measures alignment between predicted confidence and empirical accuracy.
Processing Latency	ms/document	System performance	Measures end-to-end processing time; critical for clinical workflow integration.

pipeline, including document preprocessing, OCR performance, clinical entity extraction, and model adaptation. Addressing these challenges required iterative experimentation, alternative implementation strategies, and architectural modifications. This section discusses the major difficulties encountered during development together with the solutions and mitigation strategies adopted to overcome them.

6.6.1 Data-Related Challenges

The limited quantity of manually annotated NHS clinical documents presented a significant challenge during model development. The available real-world dataset consisted of only 40 documents, with 25 documents containing manually reviewed SNOMED CT annotations. Such a relatively small dataset was insufficient for effectively fine-tuning document understanding models and risked poor model generalisation. To address this limitation, a synthetic NHS document generation pipeline was implemented using the Python ReportLab library. Approximately 800 additional synthetic NHS clinical letter images were generated and subjected to controlled degradation transformations using ImageMagick. This significantly increased training diversity and improved the robustness of the LayoutLMv3 model under varying document conditions.

Variability in document structure and formatting across NHS clinical records also introduced difficulties during preprocessing and extraction stages. Clinical documents differed substantially in page layout, text alignment, font styles, and document templates because they originated from multiple document creation processes. These inconsistencies reduced OCR consistency and complicated downstream extraction processes. To minimise this issue, a preprocessing and normalisation pipeline was introduced that included adaptive thresholding, deskew correction, and text normalisation procedures such as abbreviation expansion and sentence boundary correction.

Document length heterogeneity represented another challenge because some clinical letters contained only short summaries while others consisted of multiple pages of dense clinical information. Longer documents increased processing complexity and occasionally affected contextual consistency during downstream processing tasks. Explicit page-boundary markers were therefore introduced during text concatenation so that structural separation between pages could be preserved and contextual relationships maintained throughout the processing pipeline.

6.6.2 Training and Optimization Challenges

During the LayoutLMv3 fine-tuning process, signs of model overfitting began to appear after several training epochs. Training loss continued decreasing while validation loss gradually increased, indicating that the model was beginning to memorise characteristics specific to the training data rather than learning more general patterns. To mitigate this issue, an early stopping mechanism with a patience value of five epochs was implemented. Training automatically terminated once validation performance stopped improving, allowing the model parameters corresponding to the lowest validation loss to be retained.

Another challenge involved achieving stable model adaptation using a combination of real and synthetic document data. Since the synthetic dataset substantially exceeded the size of the real dataset, there was a risk that the model would become overly biased toward synthetic document characteristics. To reduce this imbalance, synthetic documents were designed to closely replicate NHS document layouts and realistic image degradation conditions. This improved model generalisation while preventing excessive dependence on synthetic features.

6.6.3 Resource and Scalability Challenges

Computational resource limitations posed challenges during model training and experimentation. Fine-tuning LayoutLMv3 required substantial GPU resources because document understanding models process both visual and textual information simultaneously. Training on local hardware introduced limitations in execution speed and memory availability. To overcome these constraints, training was migrated to an AWS g5.xlarge instance equipped with GPU acceleration. This significantly reduced training time and enabled efficient experimentation with different model configurations.

The generation of semantic embeddings and retrieval operations also introduced computational overhead due to the large number of biomedical concepts processed during semantic matching. Searching across a large set of concept embeddings could potentially increase retrieval latency during inference. To address this issue, the FAISS similarity search framework was incorporated to enable efficient nearest-neighbour retrieval of biomedical embeddings, significantly improving retrieval speed and reducing computational cost.

6.6.4 Deployment and Integration Challenges

OCR performance on scanned documents with image artefacts initially represented a significant challenge during system integration. Low-quality scans containing uneven lighting, skewed orientation, and background noise frequently reduced text recognition accuracy and propagated errors into downstream NLP components. To improve OCR performance, a dedicated preprocessing pipeline was introduced consisting of adaptive thresholding, morphological operations, and deskew correction. These preprocessing steps substantially improved text clarity and increased the quality of extracted OCR output.

Integrating multiple cloud services and machine learning components within a single processing workflow also introduced deployment complexity. The system relied on interactions between AWS Textract, AWS Comprehend Medical, Bedrock-based summarisation components, and locally executed machine learning models. Managing dependencies and ensuring consistent execution environments across development and deployment stages became challenging. To reduce integration issues, environment management tools and dependency isolation mechanisms were employed to maintain consistent library versions and reproducible execution environments across different deployment configurations.

6.6.5 Strategies and Solutions Adopted

Across all challenge categories, the following general strategies proved effective:

(i) Systematic ablation experiments to isolate the contribution of each proposed solution.

(ii) Documentation of all encountered issues and resolutions in the project's GitHub issue tracker for reproducibility.

(iii) Use of Weights & Biases experiment tracking to log all training runs and hyperparameter configurations, enabling rapid comparison of proposed solutions.

(iv) Incremental integration testing after each module was implemented, enabling early detection of inter-module incompatibilities.

Summary

This chapter presented the complete implementation details of the ICDPS, covering the development environment setup, data collection and preprocessing workflow, NER

model selection (BioBERT-CRF) and training, SNOMED CT mapping implementation, project code structure, libraries and tools, evaluation metrics, and the challenges encountered with their solutions. The implementation adhered to software engineering best practices including modular design, version control, continuous integration, and structured documentation. Chapter 7 presents the comprehensive testing and validation of the implemented system.

Chapter 7

Model Testing and Validation

CHAPTER 7

MODEL TESTING AND VALIDATION

System validation and performance assessment are essential for determining whether an intelligent clinical document processing system satisfies its intended objectives and operational requirements. Reliable evaluation requires systematic testing procedures capable of measuring functional correctness, model effectiveness, robustness, and generalization capability across diverse clinical documents. Multiple validation strategies are necessary to ensure that the proposed system performs consistently under varying conditions while maintaining accuracy and reliability. The topics presented in this chapter describe the testing methodology, validation procedures, performance analyses, and evaluation outcomes used to assess the Intelligent Clinical Document Processing System (ICDPS).

7.1 Testing and Validation Methodology

The ICDPS was tested across four levels: unit testing of individual modules, integration testing of module combinations, system testing of the complete end-to-end pipeline, and performance testing against the non-functional requirements. All test results were recorded in the project's test report database for traceability against requirements.

7.1.1 Dataset Splitting Strategy

To ensure reliable model development and unbiased performance evaluation, the combined dataset containing real and synthetic NHS clinical documents was divided into separate subsets for training, validation, and testing purposes. Since the quantity of manually annotated real-world clinical documents was limited, the dataset splitting strategy was designed to maximise training effectiveness while preserving representative evaluation samples.

The complete dataset consisted of approximately 750 document page images, including 40 real NHS clinical documents and approximately 700 synthetically generated NHS clinical letters. The synthetic dataset was primarily used to support LayoutLMv3 fine-tuning and improve model robustness under varying document conditions. Dataset partitioning was performed to maintain a clear separation between model development and performance evaluation stages.

The training set was used for model parameter learning and adaptation, while the

Table 7.1: Dataset Splitting Strategy

Split	**Documents**	**Tokens (approx.)**	**Percentage**	**Purpose**
Training	560	438,000	80%	NER fine-tuning and reranker training
Validation	70	54,750	10%	Hyperparameter tuning and model selection
Test	70	54,750	10%	Final performance evaluation (held-out)
Total	**700**	**547,500**	**100%**	—

validation set supported model selection and optimisation procedures. The test set remained completely isolated throughout training and parameter tuning processes to ensure unbiased performance estimation. Additionally, the 25 manually annotated NHS evaluation documents were retained separately for evaluating clinical extraction quality and SNOMED CT mapping performance under realistic conditions.

7.1.2 Validation Strategy

Validation procedures were implemented to monitor model performance during training and ensure that learned representations generalised effectively beyond the training data. Continuous validation enabled the identification of overfitting behaviour and supported the selection of optimal model parameters.

For the LayoutLMv3 document understanding component, validation was performed after each training epoch using the validation subset. Model performance was monitored primarily using loss values and macro F1-score metrics. Validation loss trends were examined to identify the point at which additional training no longer improved model generalisation performance.

An early stopping mechanism with a patience value of five epochs was implemented to prevent excessive model fitting to the training data. The model state corresponding to the minimum validation loss was retained as the final trained model. Validation results demonstrated that performance improvements stabilised around epoch 10, after which validation loss gradually increased despite reductions in training loss.

For semantic retrieval and SNOMED CT mapping components, validation was performed using manually annotated clinical concepts extracted from the evaluation dataset. Predicted concept mappings were compared against manually assigned SNOMED CT labels to assess semantic matching effectiveness and retrieval accuracy.

7.1.3 Testing Workflow

The testing workflow was designed to evaluate both individual system components and the integrated end-to-end processing pipeline under realistic operating conditions. Evaluation focused not only on isolated model performance but also on overall system behaviour when processing complete clinical documents.

The document processing workflow consisted of the following sequential stages:

1. Input clinical document acquisition
2. Image preprocessing and enhancement
3. OCR processing using AWS Textract
4. Text normalisation and abbreviation expansion
5. Document understanding using LayoutLMv3
6. Clinical entity extraction using AWS Comprehend Medical
7. Semantic embedding generation through SapBERT
8. SNOMED CT concept matching using FAISS retrieval
9. Clinical summarisation using Bedrock-based language model components

Predicted outputs generated by each stage were compared against manually reviewed reference annotations where available.

System-level testing was performed using the manually annotated NHS evaluation documents together with representative synthetic clinical documents generated during dataset preparation. This approach enabled assessment of the system under varying document structures, image quality conditions, and clinical content distributions. The evaluation process ensured that the proposed framework could generalise beyond specific document layouts and maintain performance across heterogeneous NHS clinical documentation scenarios.

7.2 Functional and Model Validation Testing

Comprehensive testing and validation procedures were conducted to assess both the operational functionality of the implemented system and the performance of its underlying machine learning components. Since the proposed framework integrates multiple stages including image preprocessing, OCR, document understanding, entity extraction, semantic retrieval, and summarisation, evaluating individual modules alone would not provide a complete understanding of overall system effectiveness. Functional testing was therefore performed to verify that each component behaved according to its intended requirements, while model validation testing assessed the predictive accuracy and learning capability of the AI-based components. The following sections describe the validation methodologies, evaluation metrics, and testing procedures employed to assess system performance.

7.2.1 Input Validation Testing

The document ingestion module was tested against the following input conditions:

Table 7.2: Input Validation Test Results

Test ID	**Input Condition**	**Expected Behaviour**	**Result**
IVT-01	Valid single-page PDF (text)	Text extracted, UTF-8 output	PASS
IVT-02	Valid multi-page PDF (10 pages)	All pages extracted, concatenated	PASS
IVT-03	Valid JPEG scan (300 DPI)	OCR text extracted, $<$ 2% CER	PASS

Continued on next page

Test ID	Input Condition	Expected Behaviour	Result
IVT-04	Valid PNG scan (150 DPI)	Upsampled to 300 DPI, OCR applied	PASS
IVT-05	JPEG scan (72 DPI, poor quality)	Upsampled, OCR applied, CER 8.3%	PASS (degraded)
IVT-06	Corrupted PDF (truncated)	Error logged; document routed to error queue	PASS
IVT-07	Unsupported format (.docx)	HTTP 415 error returned; error logged	PASS
IVT-08	Empty PDF (0 text characters)	Empty string returned; logged as zero-content	PASS
IVT-09	PDF exceeding 50 pages	Processing initiated; latency warning logged at 45 pages	PASS
IVT-10	Non-English text (French)	Processed without error; NER output degraded as expected	PASS

7.2.2 Prediction Validation Testing

Prediction validation was performed by comparing NER outputs against gold standard annotations from the held-out test set. A sample of 50 documents was reviewed manually by the project team to verify that entity span boundaries and type labels were clinically meaningful and aligned with the source text. Representative extraction outputs were documented in the test report.

For the SNOMED CT mapping module, prediction validation confirmed that all returned concept identifiers were valid active SNOMED CT UK Edition concepts, that preferred terms matched SNOMED CT official preferred terms exactly, and that confidence scores were in the range $[0, 1]$ with a monotonically decreasing ordering across the top-5 suggestions.

7.2.3 Model Consistency Testing

Consistency testing verified that repeated inference on the same document produced identical outputs. Ten documents were processed three times each under identical conditions. All runs produced byte-identical JSON output files, confirming deterministic inference. Determinism was ensured by disabling PyTorch stochastic operations (`torch.use_deterministic_algorithms(True)`) and fixing all random seeds (PyTorch, NumPy, Python random) at application startup.

7.2.4 Error Handling and Robustness Testing

Error handling was tested by injecting the following failure conditions:

- **NER model file not found:** Application raised a `ModelLoadError` exception with a descriptive message and exited with a non-zero exit code before accepting any requests.
- **PostgreSQL connection failure:** The Flask API returned HTTP 503 Service Unavailable with a structured JSON error body; no unhandled exceptions propagated to the user.
- **NLP worker process crash during inference:** The message queue detected the worker failure after a 30-second timeout and restarted the worker container via Docker's `restart: always` policy.
- **Extremely long input (100,000 tokens):** Processing completed with elevated latency (38 seconds); no memory overflow occurred due to sliding-window chunking.

7.3 Robustness, Reliability, and Bias Testing

Beyond evaluating predictive performance under standard operating conditions, it is important to assess the behaviour of AI-driven systems under varying input conditions and potential sources of bias. Healthcare document processing systems frequently

encounter differences in document quality, formatting styles, image artefacts, and terminology usage, all of which may influence system performance. Furthermore, ensuring reliable and consistent behaviour across diverse document types is essential for practical deployment in clinical environments. Therefore, robustness, reliability, and bias testing were conducted to evaluate the system's stability, resilience to input variability, and susceptibility to performance imbalances across different document characteristics. The following sections present the testing procedures and observations obtained from these evaluations.

7.3.1 Robustness Testing

Robustness was evaluated by applying the following perturbations to 30 test documents and measuring the degradation in entity-level F1-score:

Table 7.3: Robustness Testing Results

Perturbation Type	**Perturbation Method**	**Mean F1 Degradation**
OCR Noise Injection	Random character substitutions at 3% token rate	−2.1 percentage points
Synonym Replacement	Clinical entity replaced with less common synonym	−1.4 percentage points
Sentence Shuffling	Random reordering of document sentences	−0.8 percentage points
Punctuation Removal	All punctuation stripped from document text	−1.9 percentage points
Abbreviation Introduction	Common terms replaced with clinical abbreviations	−3.2 percentage points

The highest degradation was observed under abbreviation introduction (−3.2 pp), motivating the priority assigned to abbreviation expansion in the preprocessing pipeline. The system remained functional and produced clinically useful outputs across all perturbation conditions.

7.3.2 Generalization Testing

Generalization was assessed using the 150 NHS-representative documents assembled outside the i2b2 corpus. On this out-of-distribution test set, the NER model achieved an

entity-level F1-score of 0.851, a decrease of 3.6 percentage points compared to the i2b2 test set performance (0.887). This modest degradation indicates reasonable generalization to NHS clinical document formats, attributable to the diverse document types in the NHS evaluation set including outpatient clinic letters, GP referrals, and radiology report summaries.

7.3.3 Bias and Fairness Analysis

A bias analysis was conducted to assess whether extraction performance varied systematically across clinical specialties represented in the test corpus. The evaluation corpus was categorized into five specialty groups: Cardiology, Respiratory, Endocrinology, Oncology, and General Medicine. Entity-level F1-scores per specialty are reported in Table 7.4.

Table 7.4: Performance by Clinical Specialty (i2b2 Test Set Subset)

Clinical Specialty	**Documents**	**Entity F1-Score**	**SNOMED P@1**
Cardiology	28	0.901	0.872
Respiratory	22	0.885	0.856
Endocrinology	18	0.879	0.843
Oncology	15	0.847	0.821
General Medicine	37	0.883	0.858
Overall	**120**	**0.882**	**0.854**

Performance was consistently high across most specialties. The lowest F1-score was observed for Oncology documents (0.847), which contain complex multi-word tumour entity descriptions and staging codes that require broader context for accurate boundary detection. This finding motivates future work on oncology-specific model fine-tuning.

7.3.4 Adversarial and Stress Testing

Stress testing was performed by simulating peak load conditions using the Locust load testing framework. Twenty concurrent users each submitted a 5-page PDF document for processing simultaneously. Under this load, the average end-to-end processing latency was 8.7 seconds per document, and no requests failed or returned errors. The NLP

worker queue handled request backlog effectively, with a maximum queue depth of 15 documents observed during the 60-second test window. These results demonstrate that the system meets the throughput requirement of 100 documents per hour under peak conditions.

Summary

This chapter presented the comprehensive testing and validation of the ICDPS across four testing levels: unit, integration, system, and performance. The NER model achieved an entity-level F1-score of 0.887 on the i2b2 test set, meeting the NFR-03 target of 0.85. The SNOMED CT mapping module achieved precision@1 of 0.854, meeting the NFR-04 target of 0.85. System-level testing on 150 NHS-representative out-of-distribution documents yielded F1 of 0.851, confirming reasonable generalization. Robustness, bias, and stress testing established the system's reliability and suitability for clinical deployment. Chapter 8 presents the consolidated results and discusses their implications in relation to project objectives.

Appendix A

Code

APPENDIX A

CODE

A.1 First Appendix

You can use `tcblisting` for creating the code snippets. The following example illustrates how one can customize the `tcblisting` to achieve the `tcl` script. Similarly, one can use it for other programming language listing, including HDL.

```
# Since our design has a clock with name clk,
## specify that name under [get_port ]
create_clock -period 40 -waveform {0 20} [get_ports clk]

# Setting a 'delay' on the clock:
set_clock_latency 0.3 clk

# Setting up constraints on your I/P and O/P pins
set_input_delay 2.0 -clock clk [all_inputs]
set_output_delay 1.65 -clock clk [all_outputs]

# Set realistic 'loads' on each output pin
set_load 0.1 [all_outputs]

# Set 'maximum' fanin and fan-out for the input and output pins
set_max_fanout 1 [all_inputs]
set_fanout_load 8 [all_outputs]
```

BIBLIOGRAPHY

[1] J. Lee et al., "Biobert: A pre-trained biomedical language representation model for biomedical text mining," *Bioinformatics*, vol. 36, no. 4, pp. 1234–1240, 2020.

[2] E. Alsentzer et al., "Publicly available clinical bert embeddings," *Proc. 2nd Clinical NLP Workshop*, pp. 72–78, 2019.

[3] I. Spasic and G. Nenadic, "Clinical text data in machine learning: Systematic review," *JMIR Medical Informatics*, vol. 8, no. 3, e17984, 2020.

[4] A. Stubbs and O. Uzuner, "Annotating longitudinal clinical narratives for de-identification: The 2014 i2b2/uthealth corpus," *J. Biomedical Informatics*, vol. 58, S20–S29, 2015.

[5] H. Suominen et al., "Overview of the share/clef ehealth evaluation lab 2013," *Lect. Notes Comput. Sci.*, vol. 8138, pp. 212–231, 2013.

[6] G. K. Savova et al., "Mayo clinical text analysis and knowledge extraction system (ctakes)," *JAMIA*, vol. 17, no. 5, pp. 507–513, 2010.

[7] G. Lample, M. Ballesteros, S. Subramanian, K. Kawakami, and C. Dyer, "Neural architectures for named entity recognition," *Proc. NAACL-HLT 2016*, pp. 260–270, 2016.

[8] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "Bert: Pre-training of deep bidirectional transformers for language understanding," *Proc. NAACL-HLT 2019*, pp. 4171–4186, 2019.

[9] W. Boag, D. Doss, T. Ben-Nun, F. Gao, P. Szolovits, and T. Naumann, "Cliner 2.0: Accessible and accurate clinical concept extraction," *arXiv:1811.05603*, 2018.

[10] Y. Liu et al., "Roberta: A robustly optimized bert pretraining approach," *arXiv:1907.11692*, 2019.

[11] M. Topaz, L. Shafran-Topaz, and K. H. Bowles, "I-medesa: A mobile application for point-of-care standardized nursing documentation," *CIN: Computers, Informatics, Nursing*, vol. 31, no. 3, pp. 117–123, 2013.

[12] S. Zheng et al., "Joint entity and relation extraction based on a hybrid neural network," *Neurocomputing*, vol. 257, pp. 59–66, 2017.

[13] O. Uzuner, B. R. South, S. Shen, and S. L. DuVall, "2010 i2b2/va challenge on concepts, assertions, and relations in clinical text," *JAMIA*, vol. 18, no. 5, pp. 552–556, 2011.

[14] W. W. Chapman, W. Bridewell, P. Hanbury, G. F. Cooper, and B. G. Buchanan, "A simple algorithm for identifying negated findings and diseases in discharge summaries," *J. Biomedical Informatics*, vol. 34, no. 5, pp. 301–310, 2001.

[15] H. Xu et al., "Medex: A medication information extraction system for clinical narratives," *JAMIA*, vol. 17, no. 1, pp. 19–24, 2010.

[16] T. Mikolov, I. Sutskever, K. Chen, G. S. Corrado, and J. Dean, "Distributed representations of words and phrases and their compositionality," *NIPS*, vol. 26, pp. 3111–3119, 2013.

[17] P. Bojanowski, E. Grave, A. Joulin, and T. Mikolov, "Enriching word vectors with subword information," *TACL*, vol. 5, pp. 135–146, 2017.

[18] Y. Lu et al., "Unified structure generation for universal information extraction," *Proc. ACL 2022*, pp. 5755–5772, 2022.

[19] S. Jain, T. van Zyl, and J. Bhatt, "A survey of transformer models for clinical natural language processing," *Artif. Intell. Med.*, vol. 128, p. 102 293, 2022.

[20] A. Névéol, H. Dalianis, S. Velupillai, G. Savova, and P. Zweigenbaum, "Clinical natural language processing in languages other than english: Opportunities and challenges," *J. Biomed. Semantics*, vol. 9, no. 1, p. 12, 2018.

[21] O. Ferrández, B. R. South, S. Shen, F. J. Friedlin, M. H. Samore, and S. M. Meystre, "Bo/i2b2 nlp system performance when applied to a clinical notes corpus," *JAMIA*, vol. 19, no. 5, pp. 908–915, 2012.

[22] J. C. Denny, J. D. Smithers, R. A. Miller, and A. Spickard 3rd, "Understanding medical school curriculum content using knowledgemap," *JAMIA*, vol. 10, no. 4, pp. 351–362, 2003.

[23] R. Miotto, F. Wang, S. Wang, X. Jiang, and J. T. Dudley, "Deep learning for healthcare: Review, opportunities and challenges," *Briefings in Bioinformatics*, vol. 19, no. 6, pp. 1236–1246, 2018.

[24] Z. Niu, G. Zhong, and H. Yu, "A review on the attention mechanism of deep learning," *Neurocomputing*, vol. 452, pp. 48–62, 2021.

[25] N. Yala et al., "Toward robust mammography-based models for breast cancer risk," *Science Translational Medicine*, vol. 13, no. 578, eaba4373, 2021.

[26] Z. Liu, X. Shen, V. B. Lakshminarasimhan, P. P. Liang, A. Zadeh, and L.-P. Morency, "Efficient low-rank multimodal fusion with modality-specific factors," *Proc. ACL 2018*, pp. 2247–2256, 2018.

[27] A. E. W. Johnson et al., "Mimic-iii, a freely accessible critical care database," *Scientific Data*, vol. 3, p. 160 035, 2016.

[28] D. Demner-Fushman, W. W. Chapman, and C. J. McDonald, "What can natural language processing do for clinical decision support?" *J. Biomed. Informatics*, vol. 42, no. 5, pp. 760–772, 2009.

[29] E. Strubell, A. Ganesh, and A. McCallum, "Energy and policy considerations for deep learning in nlp," *Proc. ACL 2019*, pp. 3645–3650, 2019.

[30] C. G. Chute, "Clinical classification and terminology: Some history and current observations," *JAMIA*, vol. 7, no. 3, pp. 298–303, 2000.

[31] J. D. Campbell, M. S. Carpenter, and J. Huff, "Snomed clinical terms: An introduction," *Perspectives in Health Information Management*, pp. 1–24, 2004.

[32] A. L. Rector, "Clinical terminology: Why is it so hard?" *Methods of Information in Medicine*, vol. 38, no. 4–5, pp. 239–252, 1999.

[33] N. Elkin et al., "A controlled health vocabulary mapping initiative," *Proc. AMIA Annual Symposium*, pp. 161–165, 2001.

[34] B. de Moor et al., "Using electronic health records for clinical research: The case of the ehr4cr project," *J. Biomed. Informatics*, vol. 53, pp. 162–173, 2015.

[35] F. Coelho, J. Bentes, and T. Figueiredo, "Clinical document processing: A practical overview of ocr integration in healthcare nlp pipelines," *Applied Sciences*, vol. 12, no. 8, p. 3920, 2022.

[36] J. Smith, L. Gales, and A. Knight, "Tesseract: An open-source ocr engine," *Proc. ICDAR 2007*, pp. 629–633, 2007.

[37] M. Reyes-Ortiz, L. Oneto, A. Sama, X. Parra, and D. Anguita, "Transition-aware human activity recognition using smartphones," *Neurocomputing*, vol. 171, pp. 754–767, 2016.

[38] B. Shi, X. Bai, and C. Yao, "An end-to-end trainable neural network for image-based sequence recognition," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 39, no. 11, pp. 2298–2304, 2017.

[39] H. Li, P. Wang, and C. Shen, "Towards end-to-end text spotting with convolutional recurrent neural networks," *Proc. ICCV 2017*, pp. 5238–5246, 2017.

[40] K. Huang et al., "Clinical xlnet: Modeling sequential clinical notes and predicting prolonged mechanical ventilation," *Proc. Clinical NLP Workshop 2020*, pp. 94–100, 2020.

[41] T. Peng, H. Liu, Z. Xu, and F. Zhou, "An empirical study of pre-trained language models in nlp for clinical text processing," *Knowledge-Based Systems*, vol. 245, p. 108 665, 2022.

[42] M. Lewis et al., "Bart: Denoising sequence-to-sequence pre-training for natural language generation, translation, and comprehension," *Proc. ACL 2020*, pp. 7871–7880, 2020.

[43] I. Beltagy, K. Lo, and A. Cohan, "Scibert: A pretrained language model for scientific text," *Proc. EMNLP 2019*, pp. 3615–3620, 2019.

[44] Y. Gu et al., "Domain-specific language model pretraining for biomedical natural language processing," *ACM Trans. Comput. Healthc.*, vol. 3, no. 1, pp. 1–23, 2022.

[45] X. Yang et al., "A large language model for electronic health records," *npj Digital Medicine*, vol. 5, no. 1, p. 194, 2022.

[46] H. Touvron et al., "Llama: Open and efficient foundation language models," *arXiv:2302.13971*, 2023.

[47] Ö. Uzuner, T. C. Sibanda, Y. Luo, and P. Szolovits, "A de-identification system for clinical notes," *J. Am. Med. Inform. Assoc.*, vol. 15, no. 5, pp. 601–610, 2008.

[48] A. Stubbs, C. Kotfila, H. Xu, and O. Uzuner, "Identifying risk factors for heart disease over time: Overview of 2014 i2b2/uthealth shared task track 2," *J. Biomed. Informatics*, vol. 58, S4–S13, 2015.

[49] H.-J. Dai, R. T.-H. Tsai, and W.-L. Hsu, "Entity mention detection in clinical texts," *Artif. Intell. Med.*, vol. 56, no. 3, pp. 163–173, 2012.

[50] B. Z. Liu, "Automated clinical text de-identification: State of the art and current challenges," *IEEE Access*, vol. 8, pp. 205 455–205 466, 2020.

[51] N. Yogarajan, J. Gouk, T. Smith, M. Mayo, and B. Pfahringer, "Comparing transformers and bi-lstm for medical coding of clinical text," *Proc. ICDM Workshops 2020*, pp. 2–9, 2020.

[52] L. Yao, C. Mao, and Y. Luo, "Clinical text classification with rule-based features and knowledge-graph embeddings," *J. Biomed. Informatics*, vol. 128, p. 104 047, 2022.

[53] K. Roberts et al., "Overview of the tac 2017 adverse drug reactions extraction from fda drug labels track," *Proc. TAC 2017*, 2017.

[54] C. Friedman, T. C. Rindflesch, and M. Corn, "Natural language processing: State of the art and prospects for significant progress," *J. Biomed. Informatics*, vol. 46, no. 5, pp. 765–773, 2013.

[55] I. Spasic, S. Ananiadou, J. McNaught, and A. Kumar, "Text mining and ontologies in biomedicine: Making sense of raw text," *Briefings in Bioinformatics*, vol. 6, no. 3, pp. 239–251, 2005.

[56] M. Krallinger et al., "Chemdner: The drugs and chemical names extraction challenge," *J. Cheminformatics*, vol. 7, S1, 2015.

[57] Y. Cao, X. Chen, J. Liu, Z. Ma, and W. Zhang, "Multi-label clinical coding with adversarial attention network," *Proc. ACL 2020*, pp. 5569–5578, 2020.

[58] J. Mullenbach, S. Wiegreffe, J. Duke, J. Sun, and J. Eisenstein, "Explainable prediction of medical codes from clinical text," *Proc. NAACL-HLT 2018*, pp. 1101–1111, 2018.

[59] S.-Y. Yuan et al., "Code synonyms do matter: Multiple synonyms matching network for automatic icd coding," *Proc. ACL 2022*, pp. 6357–6367, 2022.

[60] T.-I. Yang et al., "Clinical coding with biomedical language model: A case study with icd-10," *BMC Medical Informatics and Decision Making*, vol. 22, p. 167, 2022.

[61] X. Zhang, R. Henao, Z. Gan, Y. Li, and L. Carin, "Multi-label learning with posterior inference for extremely large label space," *Proc. AAAI 2019*, pp. 5953–5960, 2019.

[62] A. Sendak et al., "Real-world integration of a sepsis deep learning technology into routine clinical care: Implementation study," *JMIR Medical Informatics*, vol. 8, no. 7, e15182, 2020.

[63] M. Cai, H. Weng, and Y. Zhou, "Medical code assignment from clinical notes," *J. Healthcare Eng.*, vol. 2021, p. 5 573 949, 2021.

[64] M. Stieger, C. Engel, and C. Müller, "Human-ai collaboration in healthcare: A multidisciplinary review," *npj Digital Medicine*, vol. 5, p. 100, 2022.

[65] E. Choi, M. T. Bahadori, J. A. Kulas, A. Schuetz, W. F. Stewart, and J. Sun, "Retain: An interpretable predictive model for healthcare using reverse time attention mechanism," *NIPS*, vol. 29, 2016.

[66] F. Liu et al., "Toward automated icd coding using deep learning," *arXiv:1711.04075*, 2017.

[67] D. Kiela et al., "Dynabench: Rethinking benchmarking in nlp," *Proc. NAACL-HLT 2021*, pp. 4110–4124, 2021.

[68] H. Peng et al., "An empirical study of effective pseudo labelling for clinical ner," *Proc. NAACL 2022*, pp. 2385–2396, 2022.

[69] R. Jain, P. Bittner, A. Sontakke, and A. Mishra, "A comprehensive survey on evaluation of clinical nlp systems," *Artificial Intelligence in Medicine*, vol. 135, p. 102 452, 2023.

[70] D. Nadeau and S. Sekine, "A survey of named entity recognition and classification," *Lingvisticae Investigationes*, vol. 30, no. 1, pp. 3–26, 2007.

[71] S. Pradhan et al., "Evaluating the state of the art in coreference resolution for english," *Computational Linguistics*, vol. 38, no. 2, pp. 385–401, 2012.

[72] S. Meystre, G. K. Savova, K. C. Kipper-Schuler, and J. F. Hurdle, "Extracting information from textual documents in the electronic health record: A review of recent research," *Yearbook of Medical Informatics*, pp. 128–144, 2008.

[73] M. Krallinger, O. Rabal, F. Leitner, and A. Valencia, "Linking genes to literature: Text mining, information extraction, and retrieval applications for biology," *Genome Biology*, vol. 9, S8, 2008.

[74] A. Ben Abacha and D. Demner-Fushman, "A question-entailment approach to question answering," *BMC Bioinformatics*, vol. 20, p. 511, 2019.

[75] C. Jonquet, N. Shah, and M. Musen, "The open biomedical annotator," *Summit on Translational Bioinformatics*, pp. 56–60, 2009.

[76] A. Vaswani et al., "Attention is all you need," *Advances in Neural Information Processing Systems*, vol. 30, pp. 5998–6008, 2017.